



POLITECHNIKA KRAKOWSKA im. T. Kościuszki



Wydział Inżynierii Elektrycznej i Komputerowej

Katedra Automatyki i Informatyki E-1

Kierunek studiów: Informatyka w Inżynierii Komputerowej

STUDIA NIESTACJONARNE

PRACA DYPLOMOWA

INŻYNIERSKA

Szymon Adamkiewicz

**Aplikacja mobilna przedstawiająca miejsca popularnych scen z filmów
na terenie Krakowa i okolic**

**Mobile application showing locations of popular movie scenes
in Krakow and nearby areas**

opiekun pracy:
dr inż. Damian Grela

Kraków, 2024

SPIS TREŚCI

Streszczenie pracy dyplomowej w języku polskim i angielskim.....	4
1. WSTĘP	5
1.1. Wprowadzenie	5
1.2. Cel i zakres pracy.....	6
1.3. Metodologia	6
1.4. Struktura pracy.....	6
1.5. Przegląd aplikacji o podobnym rozwiązaniu	7
1.5.1. TripAdvisor.....	7
1.5.2. ActionTrack	8
2. Technologie oraz narzędzia	9
2.1. Platformy mobilne	9
2.1.1. Android	9
2.1.2. iOS	9
2.2. Języki programowania	11
2.2.1. Flutter/Dart.....	11
2.2.2. Kotlin	11
2.2.3. Swift.....	11
2.3. Narzędzia integracji	12
2.3.1. Google Maps API	12
2.3.2. Firebase	12
2.4. Środowiska programistyczne	13
2.4.1. Visual Studio Code	13
2.4.2. Android Studio.....	13
2.4.1. Xcode	15
3. Analiza wymagań	17
3.1. Wymagania funkcjonalne	17
3.2. Wymagania нефункционалне	18
3.3. Diagram przypadków użycia	19
4. Projekt Aplikacji	25
4.1. Ekran logowania i rejestracji	25
4.2. Ekran główny oraz ekran dołączania do gry terenowej	26
4.3. Dołączanie do pokoju oraz tworzenie pokoju.....	27
4.4. Rozpoczęcie gry terenowej	28
4.5. Informacje o filmie oraz kolekcjonowanie	29
4.6. Przerwanie gry terenowej	30
4.7. Zebranie wszystkich znaczników przez użytkownika	31

4.8. Quiz.....	32
4.9. Mapa wszystkich miejsc scen filmowych.....	33
4.10. Profil użytkownika oraz ranking.....	34
4.11. Ustawienia	35
4.12. Usunięcie konta i formularz kontaktowy	36
4.13. Panel administratora	37
5. Implementacja.....	38
5.1. Wprowadzenie	38
5.2. Baza danych.....	38
5.2.1. Uwierzytelnianie	39
5.2.2. Firestore Database.....	39
5.2.3. Realtime Database	40
5.3. Biblioteki i zależności.....	41
5.4. Struktura projektu	42
5.4.1. Modele	43
5.4.2. Serwisy.....	44
5.4.3. Kontrolery	49
5.4.4. Komponenty.....	50
5.4.5. Funkcje.....	53
5.4.6. Strony.....	54
5.4.7. Wzór Haversine	56
5.5. Gra terenowa.....	57
5.6. Problemy implementacyjne	68
5.7. Testowanie	68
6. Podsumowanie	74
6.1. Podsumowanie pracy	74
6.2. Dalszy plan rozwoju aplikacji.....	74
Spis Ilustracji.....	75
Bibliografia.....	78

STRESZCZENIE

Tematem przewodnim pracy inżynierskiej jest przedstawienie projektu aplikacji mobilnej, która wpasowuje się w nowoczesne normy tworzenia systemów. Rzeczą konieczną jest omówienie podstaw w formie wstępu, jakie skłoniły do rozpoczęcia takiego projektu tj. np. przygotowana wcześniej metodologia czy cel i zakres projektu. W kolejnej części jest oparty na źródłach opis poszczególnych technologii jakie zostały użyte w projekcie lub jakie mogłyby być w alternatywnej wersji. Następny rozdział jest przeznaczony do kompletnego przedstawienia funkcjonalności aplikacji oraz wymagań a także przypadków użycia. W pracy zostanie pokazany projekt aplikacji w formie zdjęć przedstawiających różne zachowania i możliwości, których może dokonać użytkownik podczas korzystania z aplikacji przedstawiającej różne miejsca kręcenia scen filmowych w Krakowie i okolicach. Aby zobrazować proces tworzenia pomagają w tym fragmenty implementacji kodu aplikacji mobilnej wraz z opisem oraz problemy jakie wystąpiły. Całość pracy podsumują wnioski końcowe oraz obserwacje jakie zaszły podczas projektowania i implementacji aplikacji mobilnej obsługującej systemy tj. iOS, Android.

ABSTRACT

The main theme of the Engineer's Thesis is to present the design of mobile application that aligns with modern system standards. It is essential to discuss the foundational elements in the introduction, which motivated the initiation of the project, such as the pre-prepared methodology or purpose and scope of the project. The subsequent section provides a source-based description of the various technologies used in the project or those that could have been used in an alternative version. The next section is dedicated to a comprehensive presentation of the application's functionalities, requirements and use cases. The project will show the application design through images depicting various behaviors and capabilities that a user can experience while using the application, which show various movie scenes shooting in Cracow and surrounding areas. To illustrate the development process, excerpts of the mobile application code implementation along with descriptions and encountered issues are included. The entire project is concluded by final conclusion and observations made during the design and implementation of the mobile application supporting systems such as iOS and Android.

1. WSTĘP

Tworzenie ogólnodostępnych aplikacji mobilnych stało się powszechnym ułatwieniem dostępu dla różnych dziedzin ludzkiego życia tj. zarządzanie finansami, planowanie czasu, rozrywka. Dzięki wzrostowi popularności urządzeń mobilnych zwiększyło się również zapotrzebowanie na nowsze, bardziej rozbudowane i funkcjonalne aplikacje. Ułatwianie zadań oraz wykorzystywanie urządzeń do rozrywki stało się codziennością i dlatego powstaje wiele projektów czy pomysłów, które chcą wypełnić luki na rynku aplikacji.

1.1. Wprowadzenie

Rozwój technologiczny na przestrzeni lat przyczynił się do powstania pierwszych komputerów, które miały za zadanie przeprowadzać za człowieka mniej lub bardziej skomplikowane obliczenia np. równań algebraicznych. Za pierwszy komputer przyjmuje się ENIAC opracowany przez dwóch naukowców Johna Presper Eckerta i Johna Williama Mauchly'ego [1]. Następnym ważnym wynalazkiem był pierwszy prototyp telefonu komórkowego, którym była Motorola DynaTac. Urządzeniem, który rozpoczął wyścig firm do jak największego minimalizowania urządzeń przenośnych był pierwszy smartfon IBM Simon Personal Communicator [2], który nie oferował już tylko podstawowej funkcji wykonywania połączeń, ale dodatkowo umożliwiał używanie kalkulatora czy kalendarza. Z biegiem czasu smartfony stały się codziennością i posiadały już zaawansowane technologie, które porównują je do komputerów. Można powiedzieć, że w dzisiejszych czasach ludzie nie rozstają się z telefonem i oczekują nowych udogodnień w postaci aplikacji, które ułatwiają życie lub zapewniają rozrywkę. Wiele nowoczesnych aplikacji mobilnych wykorzystuje lokalizację użytkownika, aby pozyskać informację o jego położeniu i przedstawić jak najlepszą ofertę w postaci miejsc otaczających go. Stosowanie geolokalizacji jest bardzo przydatne w przypadku gier miejskich, w których znajduje się określone miejsca lub przedmioty i jest to swego rodzaju rywalizacja między użytkownikami. Technologia ta w urządzeniach mobilnych przyczyniła się do popularyzacji tzw. turystyki tematycznej, podróżnicy chcą zwiększać swój zakres wiedzy na temat miejsc, które odwiedzają. Kraków jest jednym z najpiękniejszych miast w Polsce i ma wiele do zaoferowania, dlatego właśnie dobrym przykładem jest przedstawienie miejsc, które pojawiły się w popularnych filmach czy serialach jednocześnie ukazując miejsca historyczne. Nie tylko jest to przeznaczone dla turystów, ale również może zainteresować fanatyków sztuki filmowej i będzie motywacją do odwiedzania nowych miejsc na mapie Krakowa i okolic.

1.2. Cel i zakres pracy

Celem niniejszej pracy jest zaprojektowanie oraz implementacja aplikacji mobilnej z wykorzystaniem map cyfrowych. Aplikacja powinna przedstawiać miejsca popularnych scen filmowych, z wykorzystaniem fotografii oraz legendy ze spisem informacji o filmie. Użytkownik ma mieć możliwość poruszania się po mapie Krakowa oraz okolic, posiadając lokalizacje miejsc, które może odwiedzić. Każde miejsce w swojej legendzie powinno mieć zawarte informacje tj. opis, przyporządkowany film, scenę, zdjęcie. Udostępniona powinna być również możliwość wzięcia udziału w grze terenowej, w której kilku użytkowników ma za zadanie jak najszybciej odnaleźć miejsca powiązane z np. konkretnym filmem, reżyserem czy aktorem. Aplikacja ma być dostępna na system Android oraz iOS oraz mają być wyróżnieni użytkownicy o różnej klasie uprawnień.

1.3. Metodologia

Do realizacji projektu zostanie wykorzystany pakiet narzędzi od firmy Google Flutter, który oparty jest na języku Dart. Również od firmy Google będzie pobierany klucz API do wyświetlania, modyfikowania i udostępniania użytkownikowi map cyfrowych (w przypadku projektu zostały wykorzystane mapy pochodzące od firmy Google). Zarządzanie bazą danych oraz autoryzacja użytkowników odbywać się będzie za pośrednictwem Firebase. Obsługa struktury projektowej oraz pobieranie pakietów narzędzi jest umożliwione dzięki Visual Studio Code, dodatkowo do wizualizacji używane będą emulatory urządzeń mobilnych (Google Pixel z systemem Android) które są tworzone dzięki Android Studio.

1.4. Struktura pracy

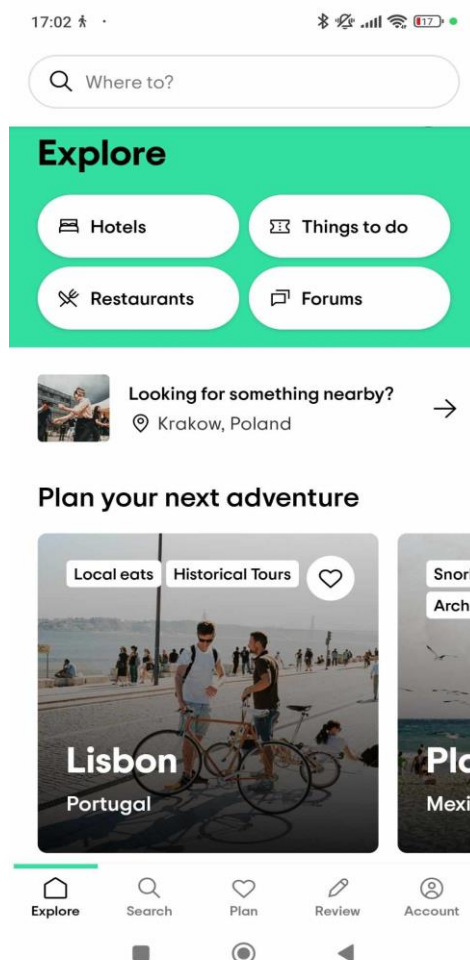
W pracy zawarte będą technologie których można używać w przypadku implementacji aplikacji mobilnych, zostaną również przedstawione wymagania stawiane projektowanej aplikacji. Opisane przypadki użycia pozwolą pokazać w jaki sposób użytkownik może korzystać z aplikacji. W kolejnym rozdziale pokazane będą możliwości jakie może mieć użytkownik korzystając z aplikacji poprzez przedstawienie wizualne projektu wraz z opisem. Ważnym elementem będzie również implementacja oraz zastosowanie wybranych do projektu technologii oraz narzędzi. Cały projekt zostanie podsumowany we wnioskach oraz zostaną opisane możliwości dalszego rozwoju stworzonej aplikacji.

1.5. Przegląd aplikacji o podobnym rozwiązaniu

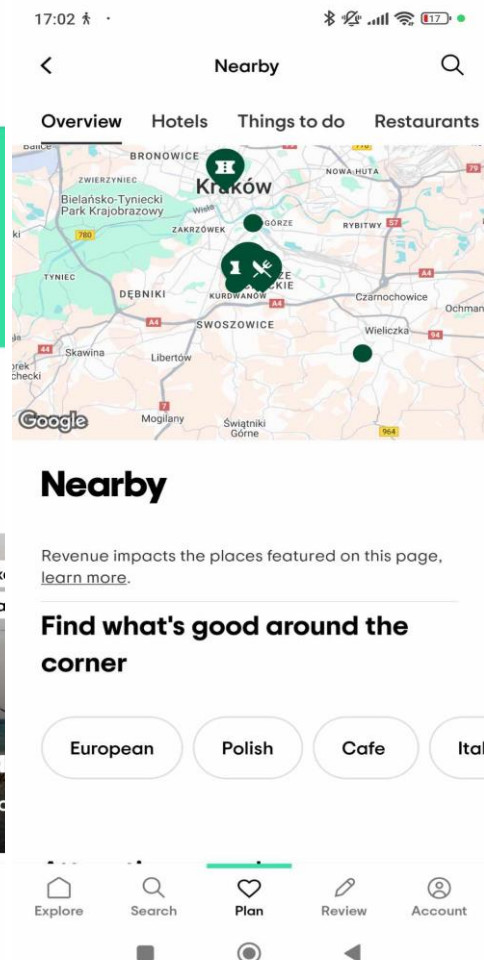
1.5.1. TripAdvisor

Tripadvisor to aplikacja wykorzystująca mapy Google, która pozwala turystom na planowanie swoich podróży od podstaw. Posiada zakładkę (Rys. 1.1), w której użytkownik może przeglądać różne miejsca na świecie i wyszukać, które warto odwiedzić w okolicy. W zakładce wyszukiwania (Rys. 1.2) w zależności od lokalizacji użytkownika lub wybranej pokazuje różne miejsca w okolicy związane z takim aktywnościami jak:

- Znalezienie noclegu w hotelu lub apartamencie, który można zarezerwować poprzez aplikację
- Atrakcje, które można odwiedzić w pobliżu od lokalizacji
- Restauracje oraz inne usługi



Rys. 1.1 Ekran główny aplikacji



Rys. 1.2 Mapa usług w okolicy

1.5.2. ActionTrack

ActionTrack to aplikacja mobilna pozwalająca na tworzenie własnych gier terenowych w danej lokalizacji. Posiada możliwość dołączenia do gry poprzez ręczne wpisanie kodu lub zeskanowanie kodu QR. Organizator rozgrywki ma możliwość tworzenia quizów, zagadek, podgląd graczy na mapie Google. Użytkownicy dostają tylko kompas wraz ze wskazaną lokalizacją następnego zadania.



Rys. 1.3 Aplikacja do tworzenia gier terenowych [3]

2. Technologie oraz narzędzia

Technologie przedstawione w niniejszym rozdziale były używane podczas tworzenia projektu. W zestawieniu brały udział języki programowania tj. Kotlin, Swift, Dart, które wspierają pisanie aplikacji mobilnych. Dodatkowo zostały opisane środowiska edytowania kodu możliwe do wykorzystania i głównie używane w powstawaniu projektu. Ważnym elementem jest również wyjaśnienie powodu wybrania bazy danych oraz narzędzi integracji oraz jaki jest ich wpływ na działanie aplikacji.

2.1. Platformy mobilne

2.1.1. Android

Android, to platforma wykorzystywana do urządzeń mobilnych tj. telefony, tablety czy innych urządzeniach przenośnych. Ma korzenie oraz powiązania z rodziną systemów Openmoko oraz MeeGo który jest połączeniem systemów firmy Nokia oraz Intel. Wszystkie wymienione systemy w tym główny, który jest opisywany, są oparte na Linuksie, czyli systemie operacyjnym, który, charakteryzuje się otwartością oprogramowania (każdy ma dostęp do kodu źródłowego oraz może wykorzystywać jego pełny potencjał). Aplikacje są pisane w języku Java oraz kompilowane za pośrednictwem interfejsów API Androida. Ważnym elementem jest integracja z różnymi elementami telefonu np. aparat, GPS co pozwala na efektywne tworzenie aplikacji. Funkcjonowanie aplikacji mobilnych składa się z kilku elementów [4]:

- Kontekst – za jego pomocą możliwe jest zarządzanie środowiskiem aplikacji w zależności od urządzenia, na którym została uruchomiona.
- Aktywność – budowanie funkcjonalności aplikacji zawierającej implementacje i pozwalające na tworzenie złożonych scenariuszy.
- Intencja – dzięki intencji możliwe jest wywołanie aktywności.
- Usługa/Serwis – ich zastosowanie nie wykorzystuje interfejsu użytkownika ani jego odpowiedzi.
- Dostawcy treści – umożliwiają dostęp do zasobów dla innych aplikacji oraz przechowywanie w bazie danych.
- Odbiorcy komunikatów – klasy opierające się na komunikacji z innymi aplikacjami.

2.1.2. iOS

iOS to programowanie systemowe dostępne tylko na urządzenia mobilne firmy Apple tj. iPhone, iPod, iPad i wiele innych. Pierwsza wersja tego systemu została wprowadzona w 2007 roku, gdzie również został przedstawiony pierwszy **iPhone**. System charakteryzuje bardzo dobry "user experience" czyli doświadczenie użytkownika w sposób wizualny czy funkcjonalny z interfejsem. Ważnym elementem jest też wydajność, która w porównaniu z systemem **Android** sprawia wrażenie bezproblemowej i posiada

mniejszą ilość awarii [5]. Niestety do publikacji aplikacji oraz skorzystania z synchronizacji z **AppleID**, które jest niezbędne do logowania się za pomocą konta Apple, konieczne jest wykupienie abonamentu w postaci opłata miesięczna lub rocznej. Dzięki opłacie dostępne są również rozszerzenia integrujące z innymi aplikacjami dostępnymi w ramach systemu **iOS**.

Tabela 2.1 Porównanie cech systemów urządzeń mobilnych [6]

	Android	iOS
Interfejs i design	Oferuje dostosowywanie interfejsu pod preferencje użytkownika za pomocą wyboru typów ekranu głównego, wielkości aplikacji, motywów czy nawigacji.	Charakteryzuje się eleganckim designem, prostym w obsłudze dzięki zorganizowaniu ikon podstawowych aplikacji systemowych.
Dostępność aplikacji	Dostępność do większości aplikacji darmowy w Google Play.	Dużo aplikacji, które wymagają płatnej subskrypcji oprócz podstawowych w App Store.
Bezpieczeństwo i prywatność	Dostępne funkcje zabezpieczeń to skanowanie linii papilarnych, rozpoznawanie twarzy czy kod 6 cyfrowy.	Wiele różnych opcji zabezpieczeń do danych telefonu tj. Face ID czy Touch ID.
Wydajność i optymalizacja	Przez popularyzację systemu co spowodowało powstawanie wiele modeli smartfonów obsługujących system Android to wydajność może się wahać i w niektórych przypadkach być bardzo niska.	Duża wydajność dzięki wykorzystywaniu systemu iOS tylko na urządzeniach należących do firmy Apple.
Przyjazność dla deweloperów	Ogólnie dostępny w przypadku projektów zewnętrznych, aplikacje w większości wpisują się w normy jakie zostały ustawione przez politykę Google Play	Ograniczenie dostępności do wykorzystywania niektórych funkcjonalności w zewnętrznych projektach, ograniczanie możliwości przez politykę firmy dotyczącą zasad panujących w App Store

2.2. Języki programowania

2.2.1. Flutter/Dart

Flutter jest to opracowany przez firmę Google framework, pozwalający na elastyczne dostosowanie interfejsu pod mobilne urządzenia. Umożliwia tworzenie aplikacji na różnych platformach tj. Android, iOS, Linux, Windows, macOS, dzięki czemu nie jest przeznaczony tylko do urządzeń mobilnych, ale również mogą być z jego pomocą tworzone aplikacje desktopowe. **Dart** jest językiem programowania, na którym zbudowany jest Flutter, sam język charakteryzuje się składnią wzorowaną na językach z rodziny C/C++ oraz posiada warstwę przeznaczoną do części wizualnej, w której znajdują się gotowe widżety i narzędzia ułatwiające budowanie interfejsu użytkownika [7]. Dzięki wykorzystaniu pakietu narzędzi projekt jest w stanie obsłużyć automatycznie dwa systemy na raz Android oraz iOS, spełniając tym główne założenie, którym było utworzenie aplikacji kompatybilnej właśnie z tymi systemami.

2.2.2. Kotlin

Kotlin jest językiem programowania funkcyjnego oraz obiektowego i posiada semantykę podobną do języka **Java**. Używa klas i metod, ale i również wspiera proceduralne wykonywanie za pomocą funkcji. Język charakteryzuje określenie statycznych typów danych, co ma służyć za ograniczanie błędów oraz awarii programów, lecz istnieje możliwość niepodawania jawnie typów. Kotlin, dzięki swojej elastyczności, jest w stanie wspierać różne języki programowania oraz sam jest przez nie wspierane. Przykładowym językiem jest JavaScript, który umożliwia przekonwertowanie kodu z Kotlinu na kod obsługiwany przez przeglądarki. Jest również wspierany przez Android co pozwala na korzystanie z narzędzi przeznaczonych do tworzenia aplikacji w tym języku [17].

Kotlin może być kompilowany do [8]:

- **JVM** - wirtualnej maszyny Javy, czyli środowiska, które jest w stanie wykonać kod bajtowy Javy pod kontrolą dowolnego systemu operacyjnego,
- **JavaScript** – obsługiwanego przez przeglądarki oraz serwery z Node.js,
- **Kod maszynowy** – który może działać na różnych systemach operacyjnych bez potrzeby wykorzystywania maszyn wirtualnych.

2.2.3. Swift

Swift, to szybki i interaktywny język programowania, za pomocą którego można tworzyć oprogramowania na urządzenia mobilne, komputery oraz inne urządzenia

umożliwiające uruchomienie kodu. Został zapoczątkowany przez firmę Apple oraz rozwijany wraz z dużą społecznością osób, które wykorzystują kod do rozwijania swojego oprogramowania, dzięki otwartemu dostępowi do kompilatora. Sposób pisanie kodu jest podobny do pisanie z wykorzystaniem łatwego języka skryptowego, tj. umożliwia podgląd wyników bez konieczności budowania i uruchamiania aplikacji. Składnia oraz zawartość bibliotek zostały skonstruowane w taki sposób, aby maksymalnie i optymalnie wykorzystać nowoczesny sprzęt. Głównymi określeniami, które charakteryzują ten język są szybkość oraz bezpieczeństwo [9].

Swift przyjmuje nowoczesne wzorce programowania i eliminuje powszechne występowanie błędów programistycznych tj. [10]:

- zmienne muszą być inicjalizowane przed ich użyciem,
- indeksy tablic nie mogą wykroczyć poza zakres,
- sprawdzanie czy liczby całkowite nie zostały przepełnione,
- automatyczne zarządzanie pamięcią,
- w przypadku awarii istnieje możliwość kontrolowanego przywrócenia poprzedniej wersji.

2.3. Narzędzia integracji

2.3.1. Google Maps API

Google for Developers oferuje wiele różnych interfejsów API oraz pakietów SDK tj. mapy, trasy, konkretne miejsca, dane środowiskowe. **Google Maps** to przedstawienie wirtualnej mapy, którą możemy dodać do projektu za pomocą publicznie dostępnego API, oferuje nam to dodatkowo różne modyfikacje. Oprócz, różnych ustawień tj. kolor mapy, znaczniki (markery), informacje na mapie, możemy również dodawać różne biblioteki usprawniające działanie naszej aplikacji i wspomaganie poruszania się użytkownika po mapie. Dodanie mapy do projektu wymaga dostępu do uprawnień użytkownika na udostępnienie lokalizacji, jest to niezbędne do namierzania jego położenia. Położenie, czyli tzw. geolokalizacja, składająca się z długości oraz szerokości geograficznej, pozwala nam na łatwe zarządzanie wartościami położenia użytkownika oraz różnych punktów, jakie chcemy umieścić na mapie.

2.3.2. Firebase

Firebase to platforma od firmy Google przeznaczona do tworzenia, zarządzania oraz utrzymywania aplikacji. Umożliwia wsparcie przy tworzeniu aplikacji na różne platformy zarówno mobilne (Android, iOS) jak i desktopowe. Oferuje uwierzytelnianie użytkowników, opcje logowania za pomocą różnych metod (np. konto Google czy Facebook), przechowuje szablony wiadomości wysyłanych w przypadku akcji, które są

skonfigurowane przez administratora. Główną zaletą platformy jest przechowywanie danych oraz aktualizacja w czasie rzeczywistym, do projektów używane są Firestore oraz Realtime Database.

Firestore jest to baza danych w chmurze NoSQL, do przechowywania i synchronizowania danych. Składa się z kolekcji, a dane dokumentów znajdujących się w kolekcji są kompatybilne z JSON. Możliwe jest wykonywanie zaawansowanych zapytań np. wyszukiwanie wektorowe.

Realtime Database również jest przechowywana w chmurze, stosuje się ją dla aplikacji, które wymagają w czasie rzeczywistym synchronizacji danych. Baza wykorzystywana jest np. dla utworzenia gry wieloosobowej, gdzie użytkownicy mogą aktualizować swoje dane na bieżąco jednocześnie mogą mieć aktualne informacje o swoim przeciwniku.

2.4. Środowiska programistyczne

2.4.1. Visual Studio Code

Visual Studio Code to edytor kodu źródłowego stworzony przez firmę Microsoft, dzięki wsparciu dla setek języków programowania jest bardzo produktywny. Edytor posiada wbudowane autouzupełnianie kodu lub stosowanie podpowiedzi za pomocą IntelliSense co usprawnia pracę przy poznawaniu nowego języka [11]. Jak większość współczesnych edytorów posiada możliwość debugowania kodu, czyli proces redukcji liczby błędów, zakładka do tego przeznaczona posiada dodatkowe informacje takie jak wartości zwracane czy czas przetwarzania funkcji. Edytor cechuje się szeroko rozwiniętą biblioteką rozszerzeń, które po pobraniu automatycznie aktualizują projekt o nowe narzędzia. Pozwala także na integrację z innymi narzędziami, obsługuje **Git**, czyli system kontroli wersji, gdzie w łatwy sposób można wykonywać operacje tj. wprowadzanie zmian, zaciąganie zmian w łatwy sposób. Dodatkowo w momencie scalania zmian edytor umożliwia podgląd poprzedniej wersji i porównuje ją z nowymi zmianami, przez co łatwo można naprawić błędy.

2.4.2. Android Studio

Do tworzenia aplikacji mobilnych na system operacyjny Android został stworzony Android Studio, który obsługuje języki tj. Kotlin, Java, czy C/C++. Edytor posiada wbudowanego menadżera wirtualnych urządzeń co pozwala na tworzenie emulatorów na różnych modelach telefonów oraz wybranie wersji systemu Android. Jest to pomocne w przypadku automatycznego testowania kodu, dzięki funkcji w **Flutter** jaką jest “hot reload” można w czasie rzeczywistym widzieć zmiany w aplikacji. Dzięki tworzeniu emulatorów możliwe jest generowanie raportu błędów i ich podgląd otrzymując komunikaty z kodem. Aplikacje budowane w edytorze są kompilowane za pomocą

systemu opartego na Gardle, który jest w stanie wygenerować wiele wariantów kompilacji, aby obsłużyć kilka urządzeń. Kiedy wystąpi problem w kompilacji istnieje analizator, który podpowiada, gdzie w projekcie występują błędy [12]. Niestety jak w przypadku każdego edytora występuje parę wad, które znacznie wpływają na tworzenie projektów. Głównym problemem są wymagania sprzętowe, narzędzie jest aktualnie przeznaczone dla komputerów, które są w stanie poradzić sobie z dużym zużyciem zasobów. Również przy dużych projektach może wystąpić długi czas budowania aplikacji.

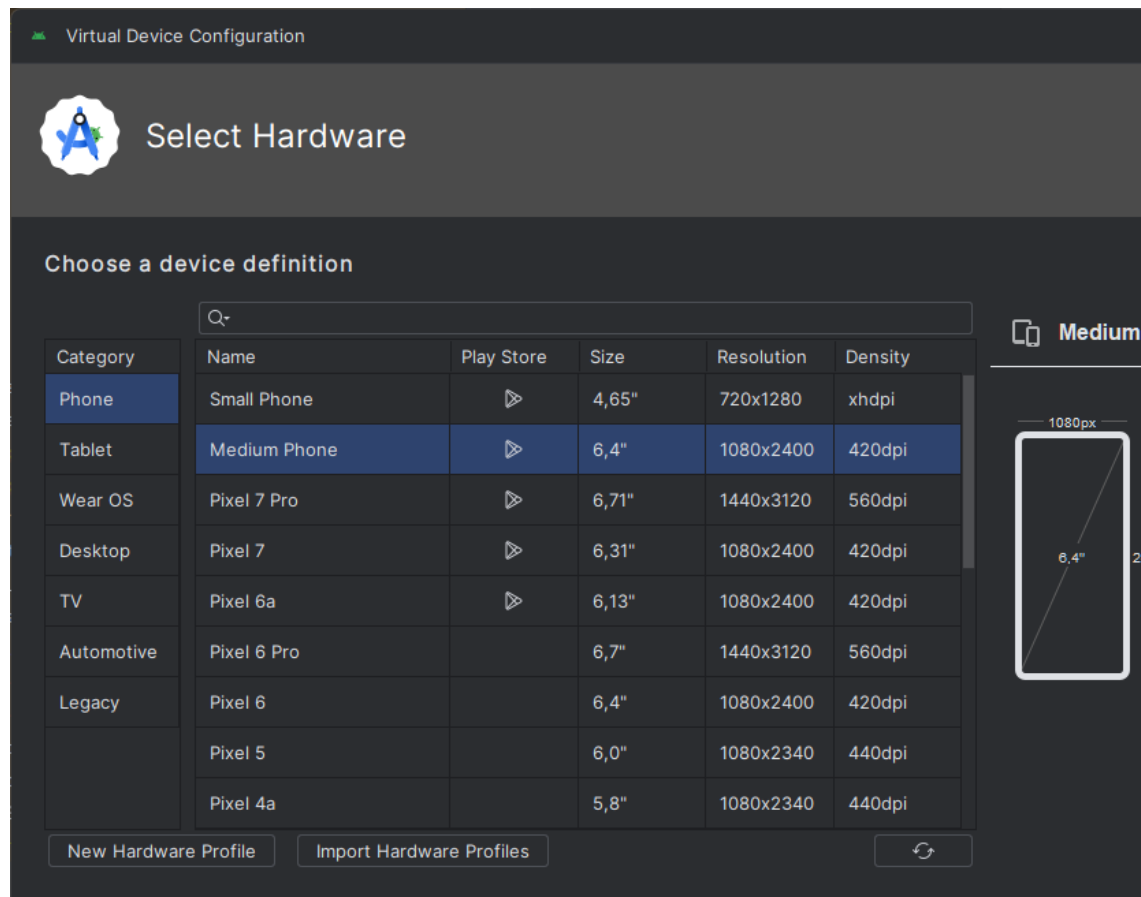
Na poniższych zdjęciach (Rys. 2.1, Rys 2.2) zostały przedstawione przykładowe modele telefonów wykorzystane do symulowania urządzeń mobilnych w chwili uruchamiania oraz podczas pracy z aplikacją.



Rys. 2.1 Google Pixel 7a

Rys. 2.2 Google Pixel 3a

Poniżej przedstawione zostały dostępne kategorie urządzeń, z których można utworzyć emulatory do korzyści projektowych tak, aby obsłużyć wszystkie konieczne platformy.



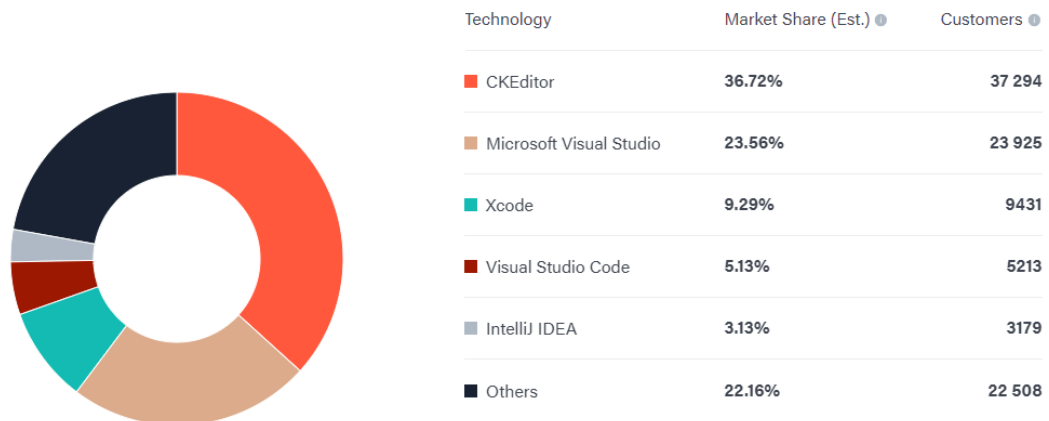
Rys. 2.3 Wyszczególnienie platform obsługiwanych przez Android Studio [13]

2.4.1. Xcode

Xcode jest środowiskiem deweloperskim przeznaczonym do pisania aplikacji na wszystkie platformy firmy Apple tj. np. iOS, iPadOS czy macOS. Jako oficjalne narzędzie posiada specjalne narzędzia do tworzenia, projektowania i publikowania aplikacji. Edytor cechuje obsługa wielu języków programowania tj. C++ czy Java, lecz głównym językiem przeznaczonym do aplikacji mobilnych jest Swift. Zaletą również jest posiadanie wbudowanego symulatora dla urządzeń Apple oraz narzędzi monitorujących prace systemu, jest to konieczne przy bieżącym testowaniu projektu [14]. Środowisko cechuje także:

- przyjazny w pracy interfejs,
- autouzupełnianie kodu,
- wiele różnych rozszerzeń ułatwiających pracę nad projektem i rozwijającym go,
- porównywanie różnych wersji zawartości plików.

Porównanie wykorzystywanych edytorów kodu oraz tekstu (Rys. 2.4) przez firmy w 2024 roku. Badanie zostało przeprowadzone na 101 550 firmach, a wykres przedstawia ilu pracowników brało w nim udział oraz jaki procent stanowi technologia do całości.



Rys. 2.4 Badanie wykorzystywania edytorów w firmach [15]

3. Analiza wymagań

Analiza wymagań określa funkcjonowanie aplikacji oraz jakie użytkownik będzie miał potrzeby podczas jej wykorzystywania. Analiza wymagań dzieli się na wymagania funkcjonalne, czyli na funkcje w systemie, które muszą zostać spełnione i zaimplementowane oraz wymagania нефункционалне czyli cechy systemu tj. np. bezpieczeństwo, zgodność [18]. W rozdziale zostanie również przedstawiony diagram przypadków użycia, aby zwizualizować użytkowanie systemu przez użytkowników z różnymi uprawnieniami. Każdy przypadek użycia posiada swój scenariusz główny oraz alternatywny, a także określone są warunki wstępne i końcowe.

3.1. Wymagania funkcjonalne

Logowanie i Rejestracja użytkownika:

- użytkownik powinien mieć wybór pomiędzy opcjami logowania tj. Logowanie za pomocą adresu e-mail oraz hasła, logowanie przez Google, logowanie przez Facebook,
- w aplikacji musi być opcja resetowania hasła za pomocą adresu e-mail,
- użytkownik musi mieć możliwość rejestracji konta za pomocą adresu e-mail oraz hasła, przez Google, przez Facebook.

Śledzenie lokalizacji użytkownika:

- w aplikacji powinna być wyświetlana informacja o aktualnym położeniu użytkownika na mapie,
- Użytkownik musi mieć informację, szczególnie podczas gry terenowej, jakie położenie ma inny użytkownik, który dołączył do tej samej gry,
- informacja o lokalizacji użytkowników powinna być aktualizowana co najmniej co minutę.

Profil użytkownika:

- użytkownik musi mieć wgląd w swoje dane profilu oraz powinien posiadać informacje o wynikach jakie uzyskał podczas gry terenowej,
- profil musi zawierać informację o odwiedzonych miejscach przez użytkownika.

Znaczniki miejsc scen filmowych:

- aplikacja powinna wyświetlać miejsca scen filmowych wraz z ich lokalizacją i krótkim opisem,
- użytkownik musi mieć informację o miejscu, które odwiedza tj. adres, tytuł filmu, gatunek, reżyser, scenarzyści, obsada oraz rok produkcji,
- aplikacja powinna umożliwiać użytkownikowi kolekcjonowanie miejsc, kiedy zbliżą się na określoną odległość, co najmniej 20m.

Bezpieczeństwo:

- użytkownik powinien mieć możliwość wysłać formularz, w przypadku wystąpienia nieoczekiwanego błędu

3.2. Wymagania niefunkcjonalne

Wydajność:

- ładowanie map Google nie powinno przekraczać kilku sekund,
- aplikacja musi być optymalna i działać płynnie na urządzeniach mobilnych,
- w systemie powinno być ograniczone użycie zapisu i odczytu lokalizacji użytkownika.

Skalowalność:

- system musi obsługiwać wielu użytkowników naraz,
- baza danych musi być w stanie przyjmować nowe wartości wciąż rosnącej liczby miejsc scen filmowych.

Bezpieczeństwo danych:

- każdy użytkownik, aby dostać się do aplikacji najpierw musi przejść proces logowania lub rejestracji,
- dane użytkowników muszą być szyfrowane i przechowywane w bezpiecznym miejscu.

Zgodność:

- aplikacja powinna działać w trybie zgodności z urządzeniami z systemem operacyjnym Android oraz z systemem iOS,
- system powinien być w stanie obsłużyć mapy np. używając Google Maps API.

Użyteczność:

- interfejs użytkownika musi być prosty w obsłudze, intuicyjny oraz zgodny z założeniami projektowymi,
- aplikacja powinna umożliwiać wsparcie w problemach z systemem poprzez wysyłanie formularza.

Niezawodność:

- system musi działać niezawodnie, obsługiwać wystąpienie błędów i przedstawiać komunikaty oraz unikać awarii.

3.3. Diagram przypadków użycia



Rys. 3.1 Diagram przypadków użycia

Przypadek użycia: Rejestracja

Aktor: Użytkownik/Administrator

Warunki wstępne: Użytkownik nie posiada jeszcze utworzonego konta w aplikacji.

Warunki końcowe: Użytkownik pomyślnie zarejestrował się do systemu

Scenariusz:

1. Użytkownik klika w napis "sign up now".
2. Pokazanie się formularza rejestracyjnego.
 - 2.a. Użytkownik wybiera opcję rejestracji przez Google.
 - 2.a.1 Pomyślnie zalogowano do systemu za pomocą konta Google, pokazanie strony głównej.
 - 2.a.2 Nie udało się zalogować, powrót do punktu 2.
 - 2.b. Użytkownik wybiera opcję rejestracji przez Facebook.
 - 2.b.1 Pomyślnie zalogowano do systemu za pomocą konta Facebook, pokazanie strony głównej.
 - 2.b.2 Nie udało się zalogować, powrót do punktu 2.
3. Wypełnienie pól formularza tj. podanie adresu e-mail, hasła oraz powtórzenia hasła.
4. Użytkownik klika w przycisk "Sign up", próba rejestracji konta.
 - 4.a. Niepoprawna rejestracja, powrót do punktu 3.

- 4.a.1 Wyświetlenie komunikatu o tym, że użytkownik błędnie wprowadził hasło, niezgodnie z polityką haseł.
- 4.a.2 Wyświetlenie komunikatu o błędnym adresie e-mail, np. błędna konstrukcja, brakujące znaki.
- 4.a.3 Wyświetlenie komunikatu o nie pasujących do siebie hasłach.
- 4.b. Poprawna rejestracja, przeniesienie użytkownika do ekranu głównego.

Przypadek użycia: Logowanie

Aktor: Użytkownik/Administrator

Warunki wstępne: Użytkownik posiada utworzone konto i nie jest jeszcze zalogowany.

Warunki końcowe: Użytkownik pomyślnie zalogował się do systemu

Scenariusz:

1. Otwarcie strony logowania
 - 1.a Użytkownik wybiera opcję logowania się przez konto Google.
 - 1.a.1 Pomyślnie zalogowano do systemu za pomocą konta Google, pokazanie strony głównej.
 - 1.a.2 Nie udało się zalogować, powrót do punktu 1.
 - 1.b Użytkownik wybiera opcję logowania się przez konto Facebook.
 - 1.b.1 Pomyślnie zalogowano do systemu za pomocą konta Facebook, pokazanie strony głównej.
 - 1.b.2 Nie udało się zalogować, powrót do punktu 1.
2. Użytkownik wypełnia formularz logowania podając e-mail i hasło.
3. Kliknięcie w przycisk "Sign in"
 - 3.a Logowanie przeszło pomyślnie, wyświetlenie strony głównej.
 - 3.b Logowanie niepoprawne.
 - 3.b.1 Niepoprawny adres e-mail, np. Użytkownik nie istnieje, powrót do punktu 1.
 - 3.b.2 Niepoprawne hasło, powrót do punktu 1.

Przypadek użycia: Wyświetlenie mapy miejsc scen filmowych

Aktor: Użytkownik/Administrator

Warunki wstępne: Użytkownik musi być zalogowany, musi zezwolić na udostępnienie lokalizacji.

Warunki końcowe: Wyświetlenie mapy Krakowa wraz z znacznikami poszczególnych miejsc scen filmowych oraz znacznikiem użytkownika.

Scenariusz:

1. Użytkownik znajduje się na stronie głównej wraz z nawigacją dolną.
2. Wybiera ikonę map i klika w nią.
3. Wyświetlenie komunikatu o udostępnieniu lokalizacji.
 - 3.a Użytkownik zatwierdza udostępnienie swojej lokalizacji.

- 3.b Użytkownik anuluje udostępnianie lokalizacji i wraca do punktu 1.
4. Wyświetlenie mapy i załadownie znaczników miejsc scen filmowych, które użytkownik może odwiedzić.

Przypadek użycia: Wyświetlenie informacji o filmie

Aktor: Użytkownik/Administrator

Warunki wstępne: Użytkownik musi być zalogowany, musi zezwolić na udostępnienie lokalizacji.

Warunki końcowe: Wyświetlenie strony z wybranym filmem i informacjami o nim.

Scenariusz:

1. Użytkownik znajduje się na stronie głównej wraz z nawigacją dolną.
2. Wybiera ikonę map i klika w nią.
3. Wyświetlenie komunikatu o udostępnieniu lokalizacji.
 - 3.a Użytkownik zatwierdza udostępnienie swojej lokalizacji.
 - 3.b Użytkownik anuluje udostępnianie lokalizacji i wraca do punktu 1.
4. Wyświetlenie mapy i załadownie znaczników miejsc scen filmowych, które użytkownik może odwiedzić.
5. Użytkownik klika w wybrany znacznik miejsca, pokazuje się nazwa oraz adres danego miejsca nad znacznikiem.
6. Użytkownik klika w wyświetloną nazwę
7. Otwiera się strona z informacjami o filmie.

Przypadek użycia: Dodanie filmu do kolekcji

Aktor: Użytkownik/Administrator

Warunki wstępne: Użytkownik musi być zalogowany, musi zezwolić na udostępnienie lokalizacji.

Warunki końcowe: Dodanie filmu do kolekcji użytkownika.

Scenariusz:

1. Użytkownik znajduje się na stronie głównej wraz z nawigacją dolną.
2. Wybiera ikonę map i klika w nią.
3. Wyświetlenie komunikatu o udostępnieniu lokalizacji.
 - 3.a Użytkownik zatwierdza udostępnienie swojej lokalizacji.
 - 3.b Użytkownik anuluje udostępnianie lokalizacji i wraca do punktu 1.
4. Wyświetlenie mapy i załadownie znaczników miejsc scen filmowych, które użytkownik może odwiedzić.
5. Użytkownik klika w wybrany znacznik miejsca, pokazuje się nazwa oraz adres danego miejsca nad znacznikiem.
6. Użytkownik klika w wyświetloną nazwę
7. Otwiera się strona z informacjami o filmie.
 - 7.a Użytkownik spełnia warunek znajdowania się w odległości maksymalnie 20m od miejsca, wyświetla się przycisk z napisem "Collect".

- 7.b Warunek nie został spełniony, użytkownik, aby móc spróbować ponownie dodać do kolekcji miejsce nagrywania sceny filmowej musi wrócić się do punktu 4.
8. Użytkownik naciska przycisk “Collect”, film wraz z informacjami dodaje się do kolekcji.

Przypadek użycia: Dołączenie do pokoju gry terenowej

Aktor: Użytkownik/Administrator

Warunki wstępne: Użytkownik musi być zalogowany

Warunki końcowe: Dołączenie do pokoju

Scenariusz:

1. Użytkownik po zalogowaniu wyświetla stronę główną z grą terenową.
 - 1.a wybiera opcje “Create room”, aby stworzyć pokój, przechodzi do punktu
 - 1.b wybiera opcje “Join room”, aby dołączyć do pokoju, wyświetla się strona z miejscem do wpisania kodu gry.
2. Użytkownik próbuje wpisać kod gry.
 - 2.a Kod został wpisany poprawnie.
 - 2.b Wpisany kod jest niepoprawny, wyświetlenie komunikatu o błędnym kodzie, powrót do punktu 2.
 - 2.c Wyświetlenie komunikatu o nieistniejącym pokoju, z powodu tego, że osoba tworząca rozgrywkę go opuściła.
3. Wyświetlenie ekranu pokoju dla dwóch użytkowników.

Przypadek użycia: Rozpoczęcie gry terenowej

Aktor: Użytkownik/Administrator

Warunki wstępne: Użytkownik musi być zalogowany, drugi użytkownik musi dołączyć do pokoju, drugi użytkownik musi zatwierdzić gotowość, użytkownicy muszą mieć zatwierdzoną opcje udostępniania lokalizacji.

Warunki końcowe: Załadowanie mapy, wraz z znacznikami do gry terenowej.

Scenariusz:

1. Wyświetlenie ekranu pokoju dla dwóch użytkowników.
2. Host gry zatwierdza gotowość i oczekuje na zatwierdzenie przez drugiego użytkownika.
 - 2.a Gra nie została rozpoczęta, ponieważ 2 użytkownik opuścił pokój, powrót do punktu 1.
 - 2.b Gra nie została rozpoczęta, ponieważ host opuścił pokój, powrót do punktu 1.
 - 2.c Gra została rozpoczęta, użytkownicy zatwierdzili gotowość.
3. Załadowanie mapy gry terenowej wraz z znacznikami miejsc do zebrania na terenie Krakowa.

Przypadek użycia: Aktualizacja rankingu użytkownika

Aktor: Użytkownik/Administrator

Warunki wstępne: Użytkownik musi być zalogowany, drugi użytkownik musi dołączyć do pokoju, drugi użytkownik musi zatwierdzić gotowość, użytkownicy muszą mieć zatwierdzoną opcję udostępniania lokalizacji.

Warunki końcowe: ukończenie quizu, aktualizacja punktów w rankingu.

Scenariusz:

1. Załadowanie mapy gry terenowej wraz z znacznikami miejsc do zebrania na terenie Krakowa.
2. Użytkownik zbliża się do lokalizacji umieszczonej na mapie.
 - 2.a Użytkownik znajduje się maksymalnie 20m od celu.
 - 2.a.1 Użytkownik naciska na miejsce lokalizacji.
 - 2.b Użytkownik jest za daleko od celu, powtarza krok 2.
3. Wyświetlenie strony z informacjami o miejscu wraz z przyciskiem “Collect”
4. Użytkownik zbiera miejsca do tymczasowej kolekcji poprzez klikanie w przycisk.
 - 4.a Użytkownik zebrał wszystkie miejsca, wyświetla się przycisk przejścia do finałowej części rozgrywki (quizu).
 - 4.b Użytkownik nie dotarł jeszcze do wszystkich miejsc, powtarza krok 4.
5. Spełnienie warunku zebrania wszystkich lokalizacji powoduje wyświetlenie przycisku kontynuacji.
6. Użytkownik klika w przycisk, wyświetlenie quizu o danej grze terenowej.
7. Użytkownik odpowiada na pytania w quizie poprzez zaznaczanie jednej z opcji oraz naciskanie przycisku “next”, aby przejść do kolejnego pytania.
8. Na końcu quizu naciska przycisk “submit” aby ukończyć grę terenową.
 - 8.a Użytkownik odpowiedział poprawnie na wszystkie pytania, wyświetlenie komunikatu o ilości punktów zdobytych oraz przycisku “finish”
 - 8.b Niewystarczająca ilość poprawnych odpowiedzi, wyświetlenie przycisku “try again”.
 - 8.b.1 Użytkownik naciska na przycisk, powrót do punktu 6.
9. Użytkownik naciska w przycisk “finish”, ukończenie gry terenowej oraz zaktualizowanie rankingu poprzez dodanie do niego uzyskanych punktów.

Przypadek użycia: Zarządzanie użytkownikami

Aktor: Administrator

Warunki wstępne: Użytkownik musi być zalogowany, musi posiadać odpowiednie uprawnienia.

Warunki końcowe: Administrator zmienia uprawnienia do aplikacji dla wybranego użytkownika.

Scenariusz:

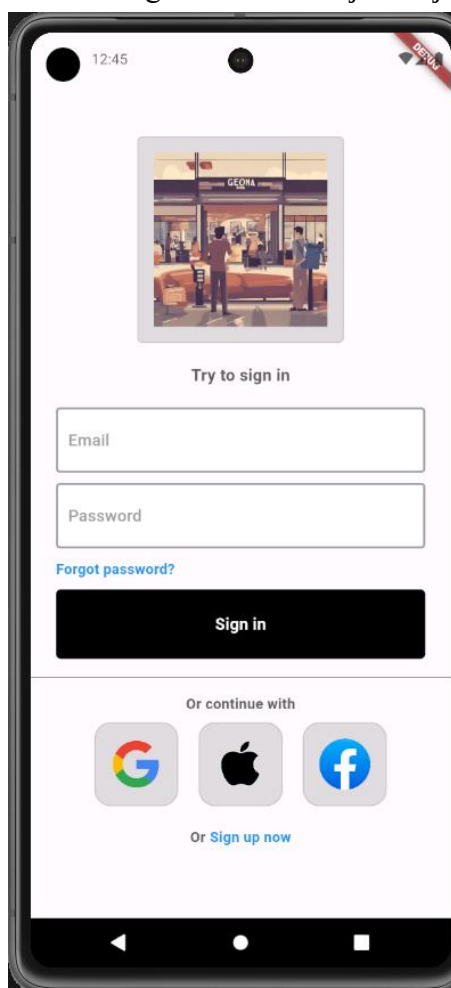
1. Wybranie z menu dolnego ikony sugerującej ustawienia aplikacji.
2. Jeśli użytkownik jest administratorem, to ostatnia pozycja w opcjach ustawień powinna być panelem administratora.
 - 2.a Użytkownik jest administratorem i klika w panel.
 - 2.b Użytkownik nie jest administratorem, opcja panelu nie jest dla niego dostępna.
3. Wyświetlenie panelu zarządzania użytkownikami, administrator naciska na zakładkę “users”.
4. Zakładka udostępnia nazwy wszystkich użytkowników wraz z możliwościami zarządzania nimi.
 - 4.a Administrator wybiera opcję nadania uprawnień administratora dla wybranego użytkownika
 - 4.a.1 Komunikat - pomyślnie zmieniono uprawnienia użytkownika.
 - 4.b Administrator wybiera opcję usunięcia konta użytkownika z aplikacji.
 - 4.b.1 Komunikat - pomyślnie usunięto użytkownika.
 - 4.b.2 Komunikat – nie można usunąć konta użytkownika, który ma wyższe uprawnienia.

4. Projekt Aplikacji

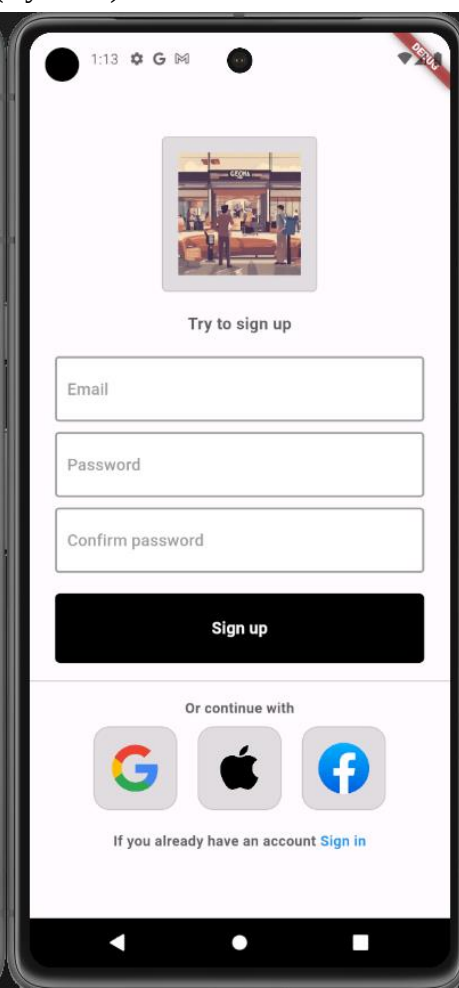
Rozdział ten jest poświęcony wizualizacji utworzonego projektu. Przedstawia widoki funkcji aplikacji dzięki czemu można pokazać w jaki sposób implementacja wpłynęła na graficzny interfejs użytkownika. Każdy ekran jest opisany poprzez wyjaśnienie jakie komponenty są funkcyjne dla użytkownika, również wyjaśnione są opcje użytkownika oraz możliwe ścieżki poruszania się po aplikacji.

4.1. Ekran logowania i rejestracji

Ekran logowania (Rys. 4.1) oferuje 3 opcje, formularz logowania przez konto Google oraz przez konto Facebook. Formularz przyjmuje wartości e-mail oraz hasło, które zostały wpisane przez użytkownika przy procesie rejestracji. Poprzez przyciski “Sign up now” i “Sign in” umieszczone na dole ekranu, użytkownik może łatwo przełączać się między widokami logowania oraz rejestracji (Rys. 4.2).



Rys. 4.1 Ekran logowania



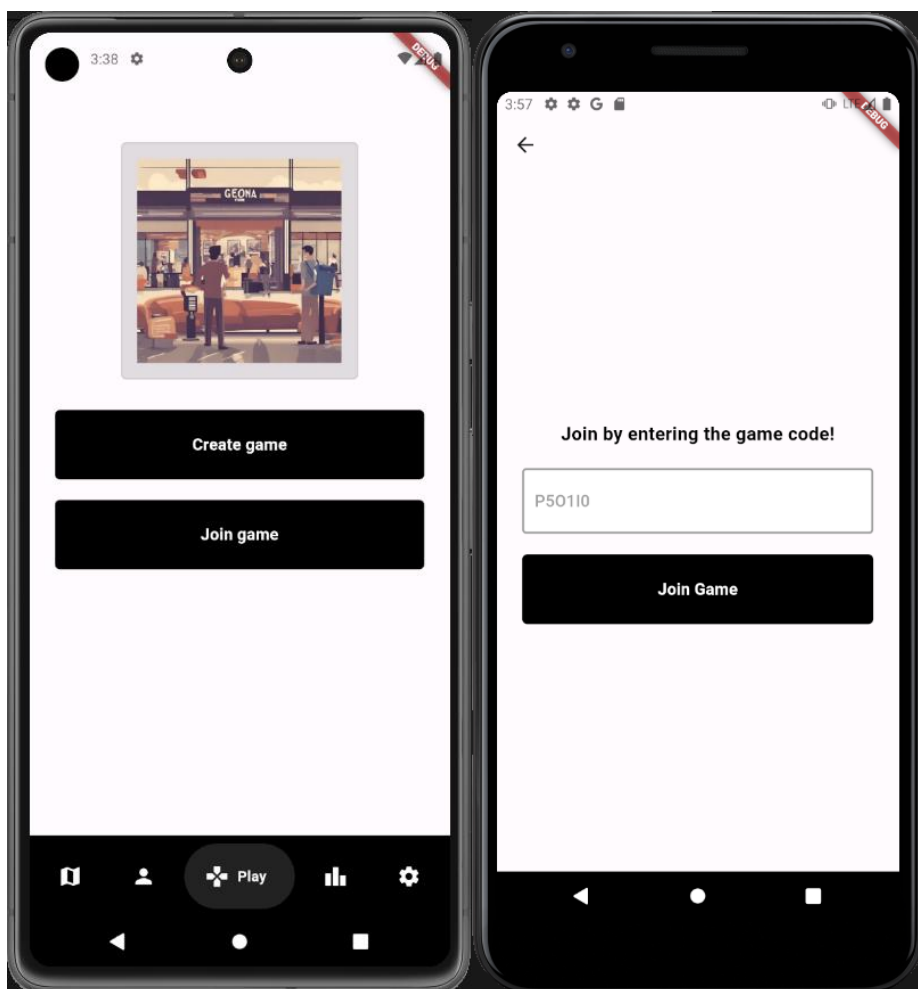
Rys. 4.2 Ekran rejestracji

Opcja logowania, rejestrowania się za pomocą Apple ID, będzie dostępna w dalszej kontynuacji rozwoju aplikacji. Formularz rejestracji pobiera od użytkownika informacje o adresie e-mail, hasło oraz jest wymagane ponowienie hasła tak, aby uniknąć błędów

wprowadzania znaków. W przypadku np. błędnego adresu e-mail lub jeśli znaki w haśle nie zgadzają się ze sobą, to na ekranie zostaną wyświetlone specjalne komunikaty odnoszące się do zaistniałego błędu.

4.2. Ekran główny oraz ekran dołączania do gry terenowej

Ekranem głównym (Rys. 4.3) jest tworzenie lub dołączanie do gry terenowej. Składa się z dwóch funkcyjnych przycisków, którymi użytkownik ma możliwość rozpocząć grę jako host od razu tworząc pokój rozgrywki, albo dołączyć do gry. Aby znaleźć aktualną sesję gry, do której użytkownik chciałby dołączyć zmuszony jest podać kod pokoju (Rys. 4.4).



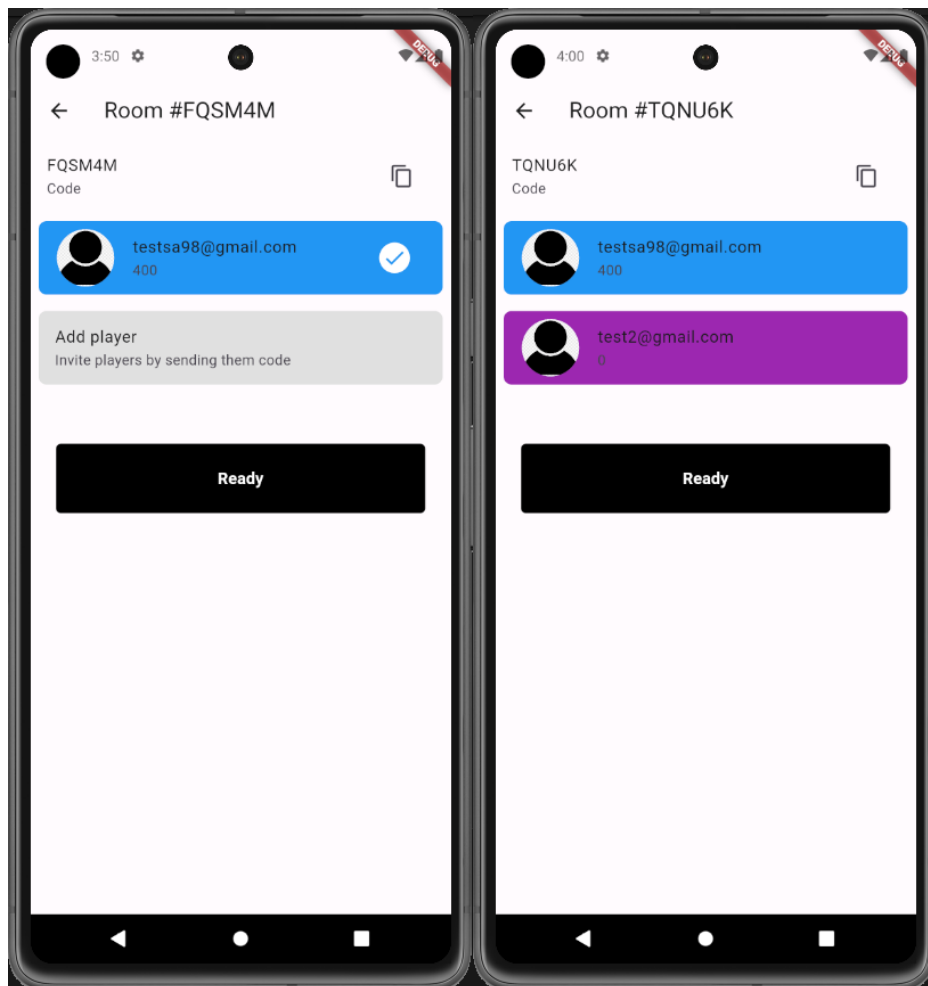
Rys. 4.3 Ekran główny

Rys. 4.4 Ekran dołączania do rozgrywki

Po kliknięciu w przycisk “Join game” ma możliwość wpisania kodu do pola tekstowego, który w następnej kolejności wyszukuje o podanym numerze sesji w bazie danych.

4.3. Dołączanie do pokoju oraz tworzenie pokoju

Użytkownik, który rozpoczął grę (Rys. 4.5) posiada możliwość skopiowania kodu pokoju, który znajduje się w górnej części ekranu klikając przycisk z ikoną “copy”. W przypadku jednego użytkownika, pozostawione jest jedno puste miejsce, w przypadku dwóch użytkowników (Rys. 4.6) są dwa elementy listy, w których znajdują się informacje tj. e-mail gracza oraz punkty w rankingu.

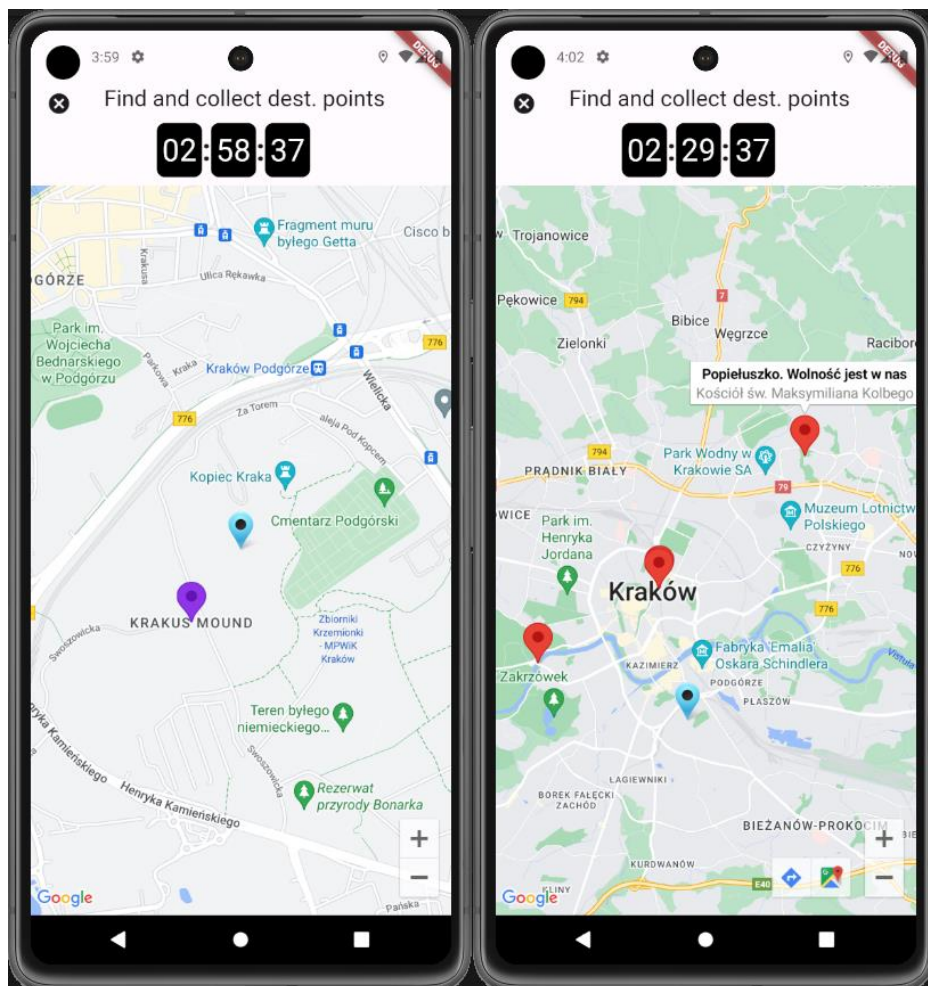


Rys. 4.5 Widok jednego użytkownika Rys. 4.6 Widok dwóch użytkowników

Kolejną funkcjonalnością jest zgłaszanie gotowości poprzez kliknięcie w przycisk “Ready”, wtedy przy elemencie listy pojawia się ikona “ready-state” (Rys. 4.5), informacja o gotowości jest przechowywana w czasie rzeczywistym w bazie danych. Kiedy dwóch użytkowników na raz zgłosi gotowość, funkcja nasłuchująca na bazę danych przeniesie automatycznie użytkowników do rozpoczynającej się gry terenowej.

4.4. Rozpoczęcie gry terenowej

Po rozpoczęciu rozgrywki dla obu użytkowników włącza się zegar z czasem na wykonanie zadania znajdującym się u góry ekranu (Rys. 4.7, Rys. 4.8). Czas jest ustalany dla każdego zestawu gier tak, aby użytkownicy byli w stanie przedostać się do każdego znacznika znajdującego się na mapie (Rys. 4.8).



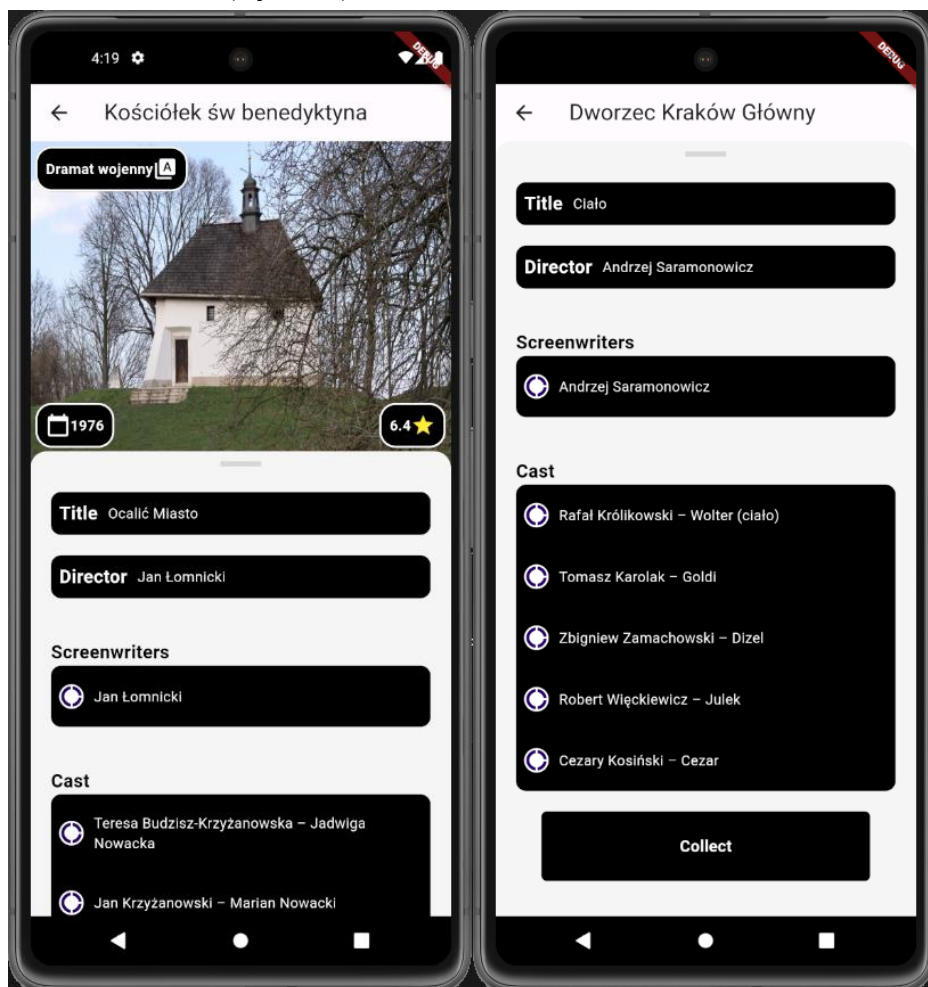
Rys. 4.7 Znacznik przeciwnika

Rys. 4.8 Znaczniki miejsc

Każdy z użytkowników posiada swój znacznik z lokalizacją, który nasłuchuje na zmianę lokalizacji oraz znacznik przeciwnika (Rys. 4.7) w kolorze fioletowym, którego położenie aktualizuje się co minutę. Każdy znacznik posiada możliwość podglądu informacji o filmie, a każdy zestaw gry jest przypisany do konkretnego reżysera, aktora lub scenarzysty. Informacje o filmach mają pomóc użytkownikowi wyciągnąć wnioski na temat osoby, o której w części finalnej rozgrywki będzie quiz.

4.5. Informacje o filmie oraz kolekcjonowanie

Głównym elementem strony jest zdjęcie przedstawiające miejsce, w którym była nagrywana scena filmowa (Rys. 4.9).



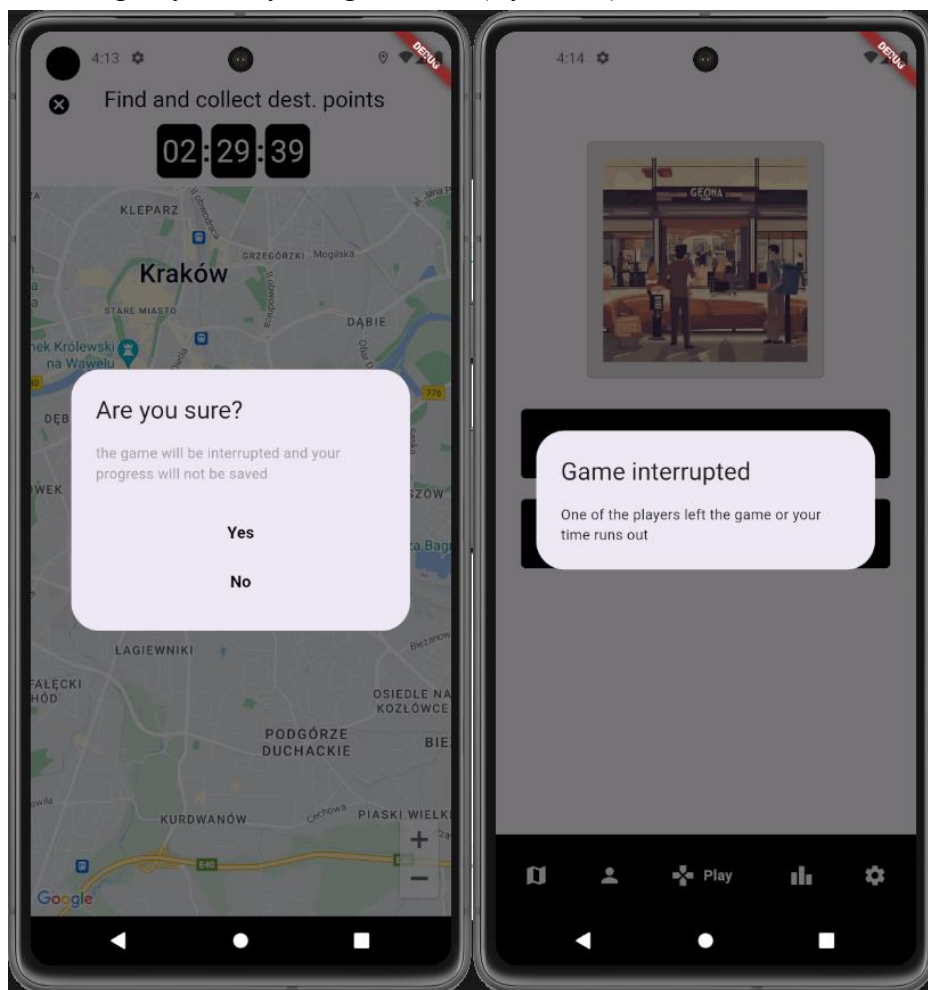
Rys. 4.9 Informacje o filmie

Rys. 4.10 Wyświetlenie przycisku

Na zdjęciu znajdują się dodatkowe informacje o znalezionej średniej ocenie przez krytyków filmowych, gatunku filmowym oraz roku produkcji. W dolnym kontenerze znajdują się szczegółowe informacje tj. tytuł, imię i nazwisko reżysera, scenarzyści oraz obsada. Jeżeli użytkownik spełni warunek odległości od jego położenia do miejsca docelowego, to wyświetli się przycisk “Collect” (Rys. 4.10) umożliwiający dodanie filmu do kolekcji. W nagłówku strony znajduje się również nazwa ulicy lub nazwa miejsca, w którym była nagrywana dana scena filmowa oraz przycisk powrotu do mapy.

4.6. Przerwanie gry terenowej

Użytkownik ma możliwość zrezygnować z gry terenowej poprzez kliknięcie w przycisk “x” w górnym lewym rogu ekranu (Rys. 4.11).

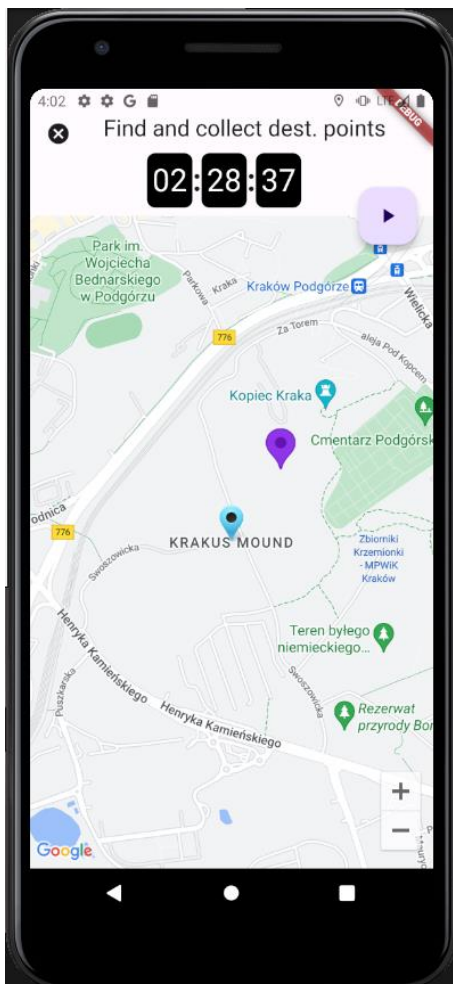


Rys. 4.11 Komunikat do zatwierdzenia Rys. 4.12 Komunikat o opuszczeniu rozgrywki

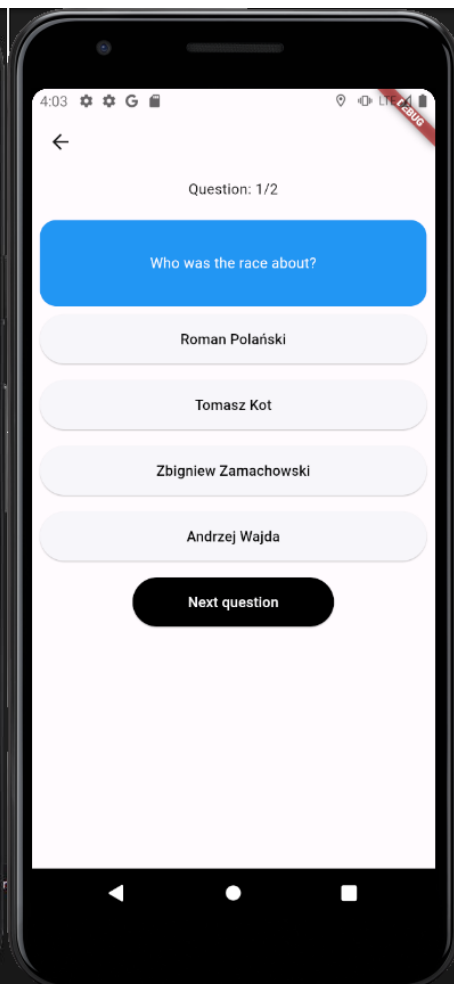
Po kliknięciu pojawi się komunikat czy jest zdecydowany na rezygnację oraz przedstawia jakie będą konsekwencje, użytkownik musi potwierdzić lub odrzucić komunikat. Głównym skutkiem przerywania rozgrywki jest brak możliwości otrzymania jakichkolwiek punktów do rankingu dla obu użytkowników. Kiedy rezygnacja zostanie potwierdzona zostanie wyświetlony komunikat (Rys. 4.12) o opuszczeniu rozgrywki przez jednego z graczy.

4.7. Zebranie wszystkich znaczników przez użytkownika

W przypadku, kiedy użytkownikowi udało się zdobyć wszystkie lokalizacje, wyświetli się przycisk kontynuacji (Rys. 4.13) w górnym prawym rogu, który nasłuchuje na zmianę w kolekcji miejsc użytkownika. Zdobywanie miejsc odbywa się poprzez zbliżenie się na co najwyżej 20 m od celu oraz kliknięcie w przycisk “Collect” (Rys. 4.10), przy wszystkich lokalizacjach z zestawu gry.



Rys. 4.13 Przycisk kontynuacji

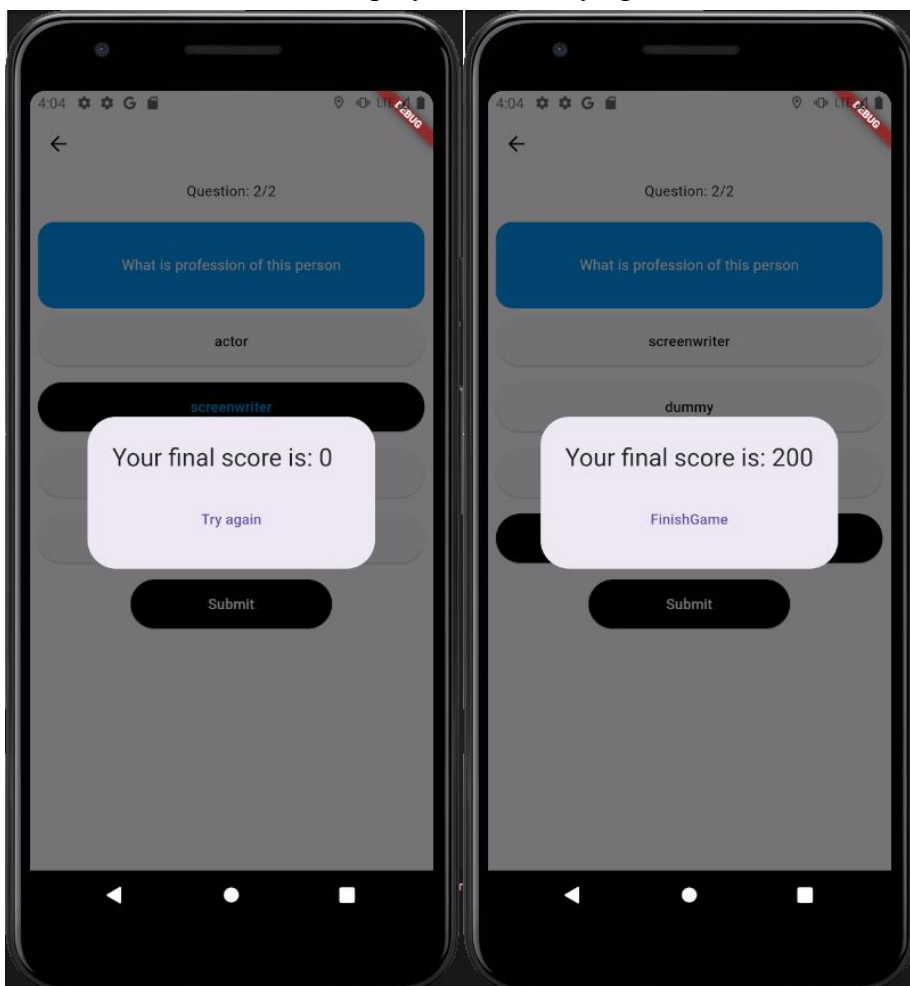


Rys. 4.14 Ekran startowy Quizu

Przycisk przenosi użytkownika do ekranu z quizem (Rys. 4.14) adekwatnym do zestawu gry, który został wcześniej utworzony na powtarzających się nazwiskach reżyserów, scenarzystów, aktorów w kolekcji filmów. Użytkownik musi odpowiedzieć na pytania związane z artystami, w niebieskim nagłówku znajduje się pytanie na temat, a pod nim 4 możliwe odpowiedzi.

4.8. Quiz

Quiz posiada 2 rodzaje stanów: ukończony oraz nieukończony. Jeżeli użytkownikowi udało się poprawnie odpowiedzieć na wszystkie pytania, to pojawi się komunikat (Rys. 4.16) z wynikiem w postaci punktów oraz z przyciskiem “Finish Game”. W drugim przypadku użytkownik otrzyma komunikat (Rys. 4.15) z niewystarczającą ilością punktów lub nawet ich brakiem oraz z przyciskiem “Try again”.



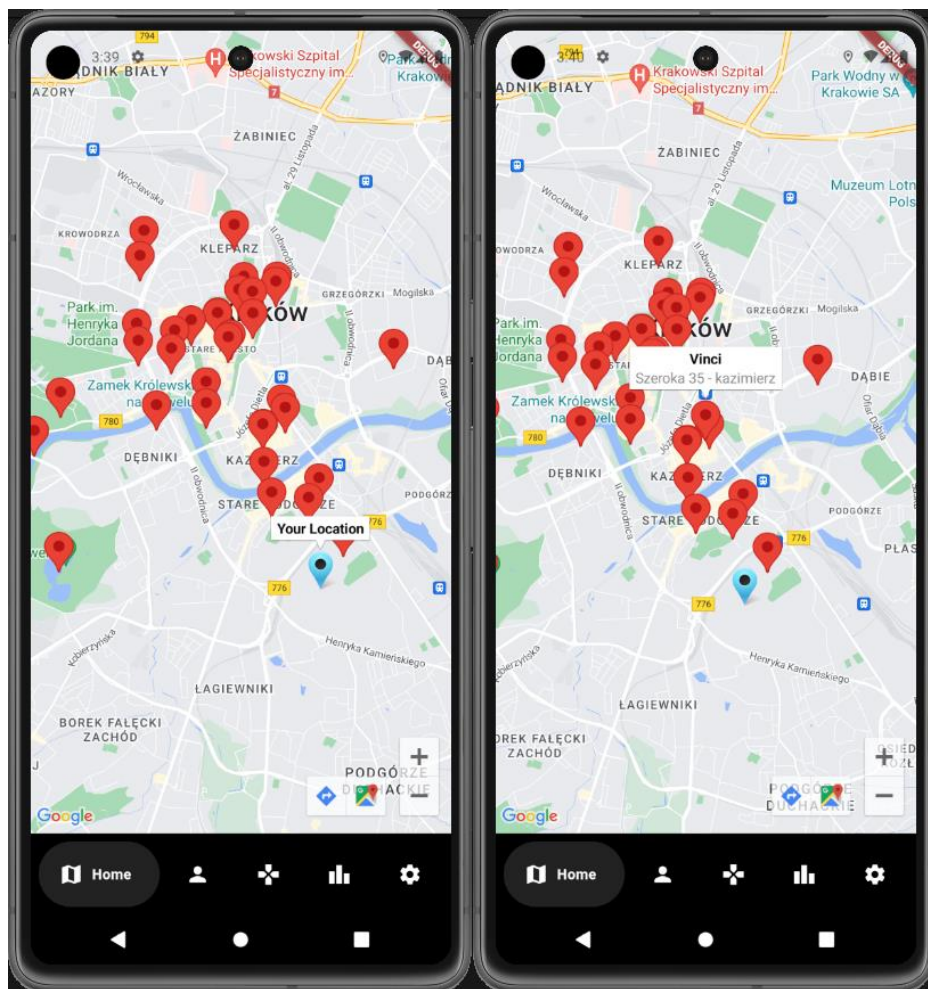
Rys. 4.15 Komunikat spróbuj ponownie

Rys. 4.16 Ukończenie quizu

Kliknięcie przycisku z przypadku pierwszego spowoduje zakończenie gry dla obu użytkowników, zaktualizowanie rankingów oraz przejście do ekranu głównego. Kliknięcie przycisku z przypadku drugiego przeniesie użytkownika z powrotem do mapy gry terenowej tak, aby mógł ponownie poszukać informacji na temat filmów, a następnie umożliwia mu przystąpienie ponowne do quizu. Gra trwa, dopóki jeden z graczy nie ukończy quizu odpowiadając poprawnie na wszystkie pytania lub jeśli skończy się określony czas na zegarze.

4.9. Mapa wszystkich miejsc scen filmowych

Mapa jest dodatkową funkcjonalnością dla użytkownika, która umożliwia podgląd wszystkich miejsc, w których nagrywane były sceny filmowe. Użytkownik posiada informację o swojej lokalizacji (Rys. 4.17) i może ją śledzić w czasie rzeczywistym. Istnieje również możliwość kolekcjonowania miejsc, stosując zasadę odległości od celu, czyli 20 m. Zebranie znacznika odbywa się poprzez kliknięcie w wybrany marker, a następnie nazwę filmu (Rys. 4.18).

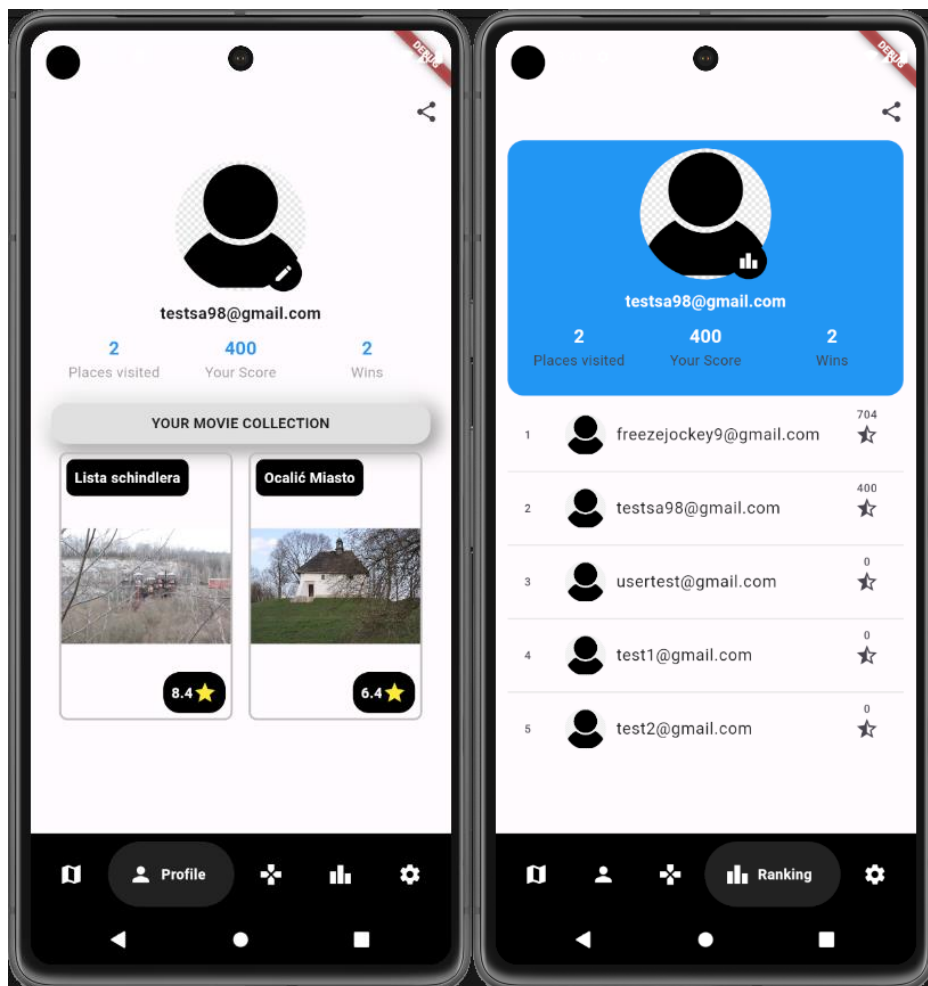


Rys. 4.17 Lokalizacja użytkownika Rys. 4.18 Znacznik z informacją o filmie

Wykonane czynności przeniosą użytkownika do strony informacji o filmie (Rys. 4.9), a tam będzie dostępny przycisk dodania do kolekcji (Rys. 4.10). Wszystkie zebrane znaczniki będą pojawiały się w profilu użytkownika.

4.10. Profil użytkownika oraz ranking

Profil użytkownika (Rys. 4.19) zawiera informacje tj. Adres e-mail, liczba odwiedzonych miejsc, wynik punktowy w rankingu oraz liczba wygranych gier. Poniżej znajduje się kolekcja filmów zebrana przez użytkownika, każdy z elementów przenosi do informacji o danym filmie.



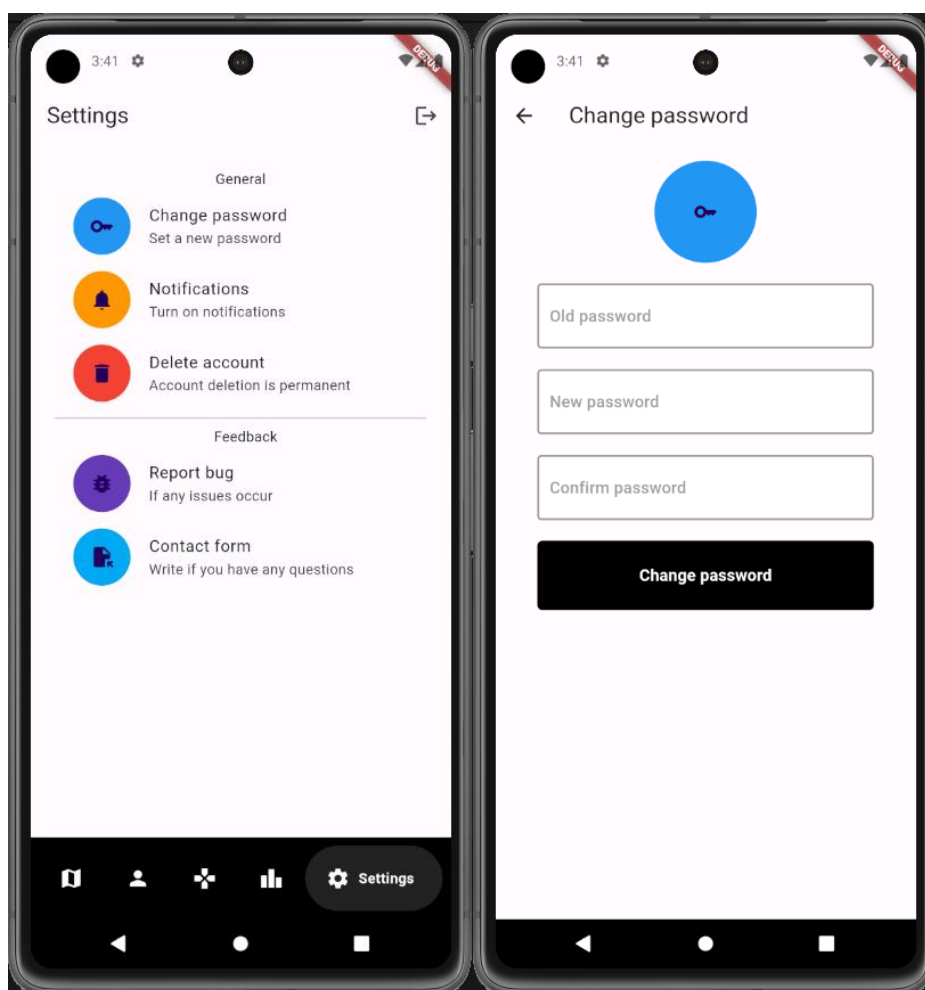
Rys. 4.19 Profil użytkownika

Rys. 4.20 Widok rankingu

Ranking (Rys. 4.20) jest przedstawieniem listy użytkowników i ich pozycji, również w rankingu znajdują się te same informacje co w głównej części profilu użytkownika.

4.11. Ustawienia

Ekran ustawień (Rys. 4.21) jest częścią aplikacji, która będzie rozwijana, użytkownik posiada tylko możliwość zmiany hasła, usunięcie konta czy wysłanie formularzy kontaktowych. Na przedstawionym widoku znajdują się wylistowane opcje dla użytkownika oraz przycisk w górnym prawym rogu, którym można wylogować się z systemu. W przypadku, gdy użytkownik będzie chciał zmienić swoje hasło może to zrobić za pomocą funkcji “Change password” (Rys. 4.22). Musi podać informacje w przeznaczonych do tego polach tekstowych tj. poprzednie hasło (zwiększenie bezpieczeństwa w przypadku niepożądanego dostępu do aplikacji), nowe hasło oraz jego potwierdzenie.



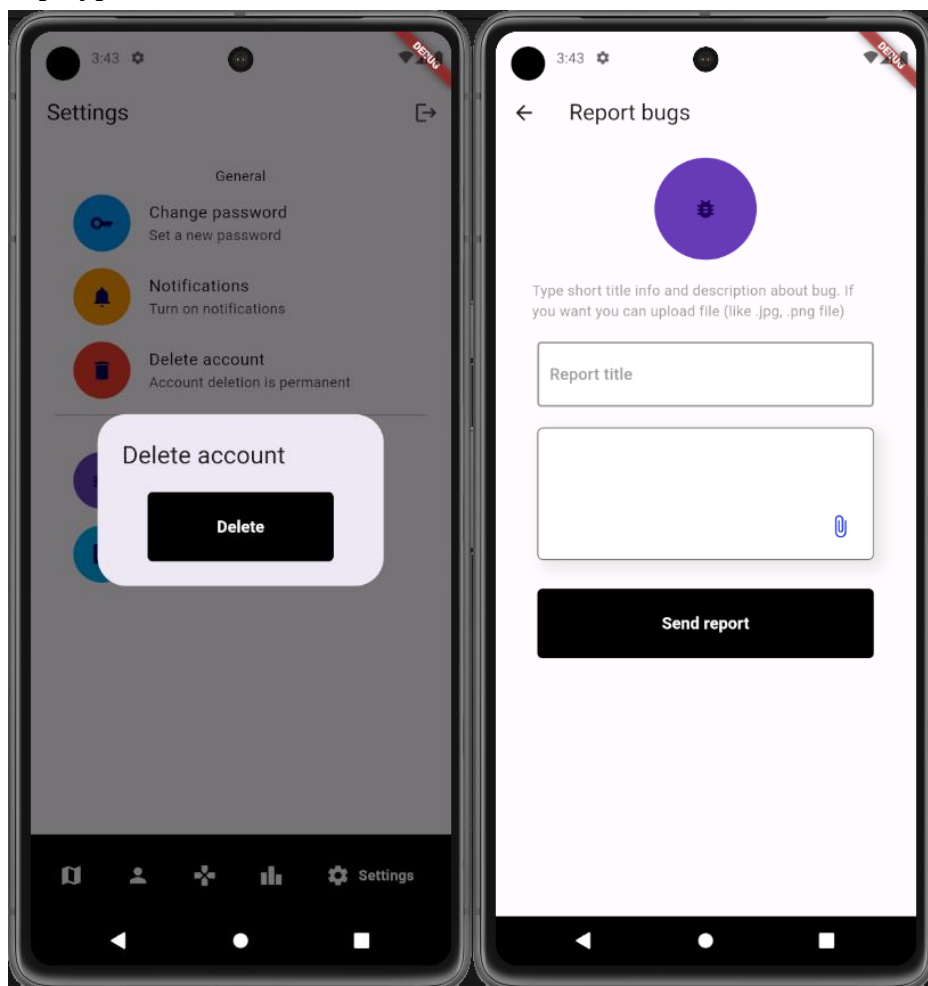
Rys. 4.21 Ekran ustawień

Rys. 4.22 Formularz zmiany hasła

Kiedy pojawi się jakiś błąd np. poprzednie hasło zostało wprowadzone niepoprawnie, to zostanie wyświetlony komunikat o błędach w formularzu.

4.12. Usunięcie konta i formularz kontaktowy

Użytkownik posiada możliwość usunięcia swojego konta za pomocą funkcji “Delete account”, spowoduje to nieodwracalną utratę dostępu do konta. Po naciśnięciu na funkcję wyświetli się komunikat z przyciskiem “Delete” (Rys. 4.23), po kliknięciu na niego pojawi się komunikat podobny do (Rys. 4.11) tak, aby użytkownik nie usunął swojego konta przez przypadek.



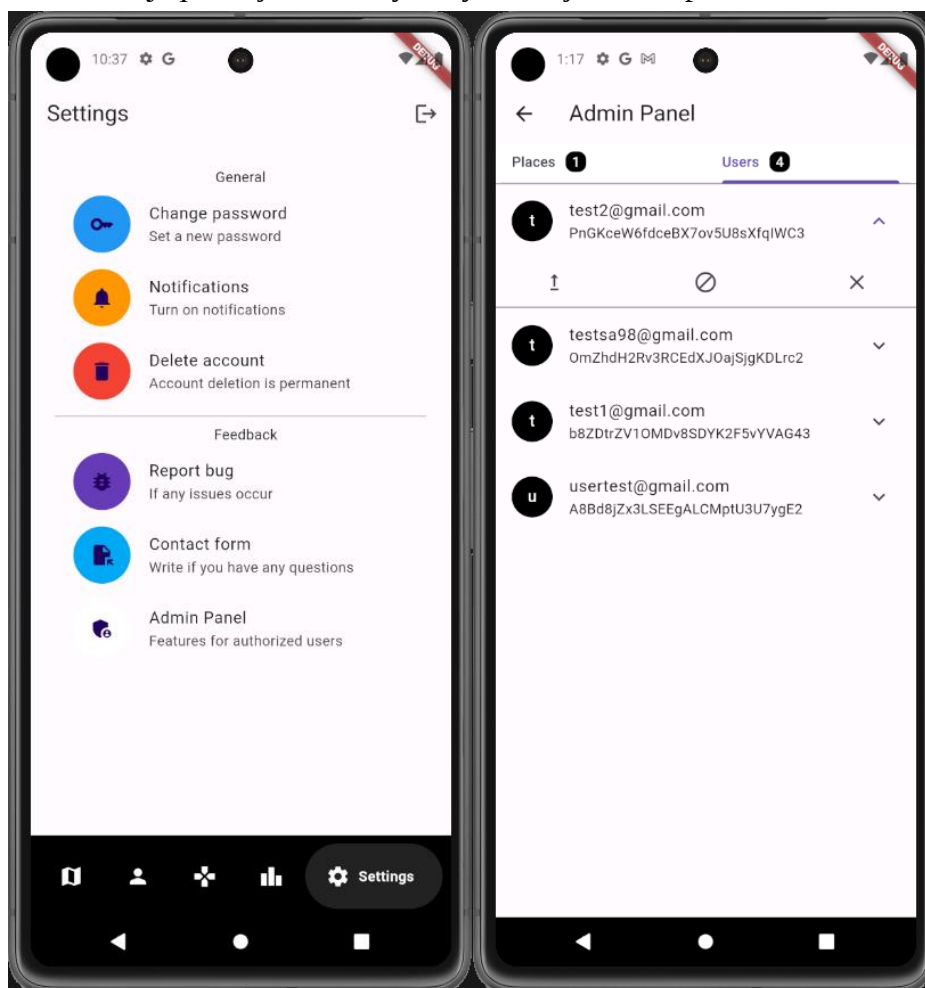
Rys. 4.23 Usunięcie konta

Rys. 4.24 Formularz zgłoszenia błędu

Kolejną możliwością jest wysyłanie formularzy, jednym z nich jest wysyłanie informacji o błędach w systemie, czyli funkcja “Report bugs” (Rys. 4.24). Użytkownik podaje tytuł błędu jaki napotkał, poniżej znajduje się większe pole tekstowe, w którym może podać scenariusz zaistnienia błędu, a nawet dodać plik (dostępne różne format) dla zobrazowania sytuacji.

4.13. Panel administratora

Administrator jest to typ użytkownika z wyższymi uprawnieniami, posiada dostęp do panelu, który znajduje się w ustawieniach (Rys. 4.25). Panel administratora (Rys. 4.26) składa się z dwóch funkcji tj. przyjmowane formularze, zarządzanie użytkownikami. Częścią rozwojową aplikacji będą formularze, które wysyła użytkownik, będą to informacje na temat filmów nagrywanych w Krakowie. Administrator będzie mógł je zatwierdzać lub odrzucać, będzie to forma udostępnienia użytkownikom możliwości wpływania na rozwój aplikacji oraz bazy danych miejsc na mapie.



Rys. 4.25 Ustawienia Administratora Rys. 4.26 Panel administratora

Główną funkcjonalnością, jaką zajmuje się administrator, jest zarządzanie użytkownikami, dostępne jest nadanie uprawnień użytkownikowi, zablokowanie użytkownika, usunięcie konta użytkownika. Zakładka "Users" zawiera listę wszystkich użytkowników z wyłączeniem administratorów oraz przyciski opcji jakie można wykonywać na elemencie listy.

5. Implementacja

Dział w którym przedstawiony zostaje sposób podejścia do problemu jaki został założony na początku tworzenia projektu. Implementacja jest to napisanie kodu, który w poprawny sposób wpływa na działanie aplikacji i opisuje jak funkcje, których może używać użytkownik zostały zaprojektowane. W rozdziale został ukazany sposób wykorzystania bazy danych, wybranego języka programowania oraz innych technologii opisanych w rozdziale drugim. Ważnym elementem jest przedstawienie synchronizacji użytych narzędzi i umiejętność utworzenia funkcji wykorzystujących dane technologie.

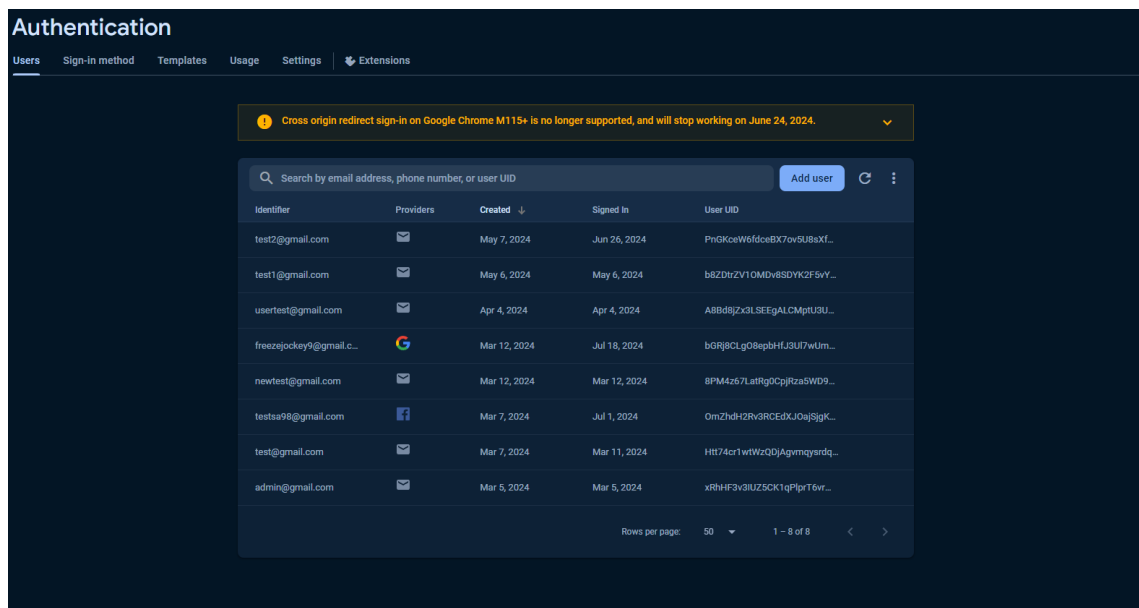
5.1. Wprowadzenie

Aplikacja została zbudowana z wykorzystaniem języka programowania Dart oraz zestawu narzędzi Flutter. Jako bazę danych wybrany został Firebase, który oferuje w łatwy i bezpieczny sposób przechowywanie zasobów aplikacji. Aby korzystać z danych w czasie rzeczywistym zostały wykorzystane dwie formy zapisu i pozyskiwania danych, czyli przez dostęp do dokumentów oraz dostęp do referencji. Do uzyskania map oraz możliwości ich wykorzystywania został użyty klucz API Google Maps. Kod aplikacji został pogrupowany na kilka części tj. serwisy, funkcje powtarzające się, strony, modele czy kontrolery co pozwala na przejrzystą strukturę.

5.2. Baza danych

Firebase jest to dobry wybór bazy danych noSQL, kiedy chcemy w szybki, bezpieczny sposób oraz w czasie rzeczywistym przechowywać, odczytywać czy modyfikować dane. Baza działa w chmurze i oferuje szereg różnych funkcjonalności.

5.2.1. Uwierzytelnianie

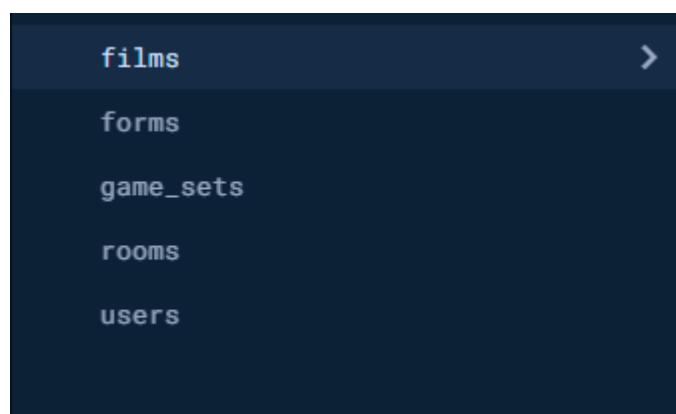


Rys. 5.1 Panel uwierzytelniania jako jedna z funkcjonalności Firebase

Zakładka “Authentication” umożliwia włączanie opcji w jakie użytkownik może się zalogować do Aplikacji. Wszystkie informacje o zarejestrowanych użytkownikach są przechowywane w bazie w zakładce „Users”(Rys. 5.1), a włączonymi opcjami logowania są logowanie z Google, Facebookiem oraz za pomocą loginu i hasła. W zakładce “Templates” można ustawić gotowe wzorce wiadomości jakie mają być wysyłane na konkretny adres email w przypadkach tj. np. weryfikacja adresu email, resetowanie hasła. Projekt obejmuje dokładnie te dwa typy wysyłanych wiadomości.

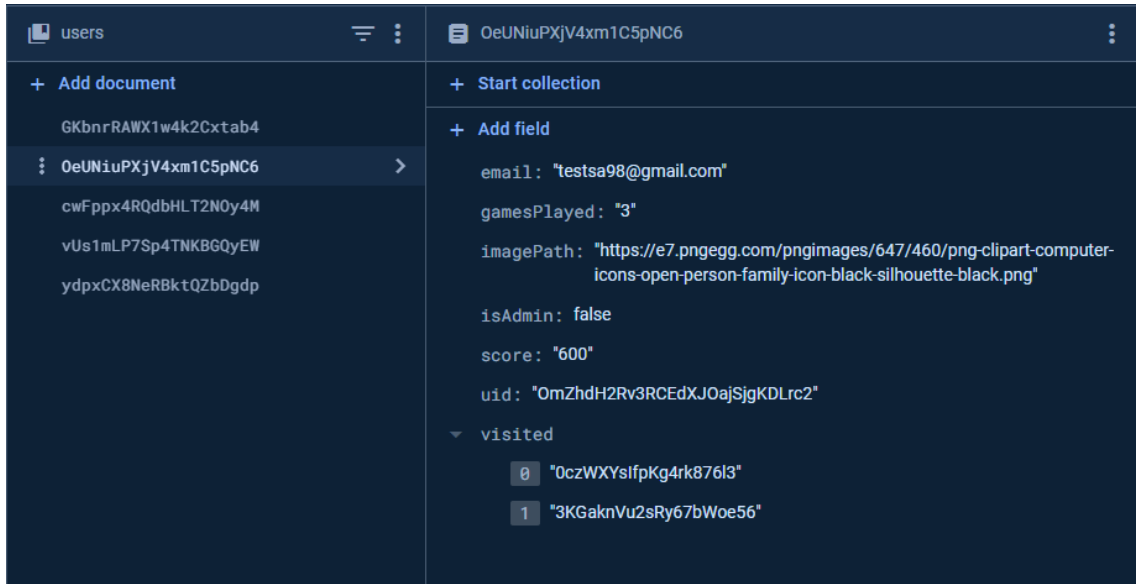
5.2.2. Firestore Database

Do przechowywania danych, używane są kolekcje “Firestore Database”, które zapisywane są w postaci dokumentów. Każdy z dokumentów składa się z pól, a pola można ustawić jak różny typ danych.



Rys. 5.2 Struktura kolekcji w Firestore

Przedstawiona struktura (Rys. 5.2) jest złożona z różnych kolekcji dokumentów. Każda kolekcja posiada dokumenty z różnymi polami oraz typami danych, niektóre np. “users” i kolekcja “rooms” są powiązane przez przechowywanie numeru identyfikującego użytkownika.



Rys. 5.3 Przykładowa struktura zbudowanego dokumentu w kolekcji users

Utworzenie kolekcji “users” (Rys. 5.3) może wydawać się powtórzeniem przechowywania użytkowników, za co odpowiedzialna jest funkcjonalność uwierzytelnianie. Różnicą jest, że uwierzytelnianie przechowuje informacje o danych logowania użytkownika a w kolekcji zapisywane są informacje występujące podczas używania aplikacji. Na przykład ilość gier zagranych przez użytkownika, informacja o roli użytkownika w aplikacji czy lista odwiedzonych miejsc.

5.2.3. Realtime Database



Rys. 5.4 Struktura referencji zawartych w bazie RealtimeDatabase

Baza danych w czasie rzeczywistym jest używana do przechowywania danych tymczasowych i ułatwiających aktualizowanie informacji w aplikacji. Podzielona jest na pokoje oraz użytkowników (Rys. 5.4). Kiedy użytkownik rozpoczyna rozgrywkę tworzy pokój a baza danych przyjmuje wartość jego numeru identyfikującego, czyli ciąg znaków który znajduje się np. przy kluczu “player1_uid”. W momencie dołączenia drugiego użytkownika to wartość drugiego klucza otrzymuje wartość jego numeru identyfikującego. Dla każdego użytkownika, dla jego wartości “uid” przechowywane są informacje tj. zebrane miejsca podczas rozgrywki, aktualna lokalizacja użytkownika, gotowość w postaci wartości bool.

5.3. Biblioteki i zależności

W celu utworzenia funkcjonalnej aplikacji, niezbędne było wykorzystanie zewnętrznych ogólnodostępnych bibliotek oraz pakietów narzędzi (Rys. 5.5).

```
cupertino_icons: ^1.0.2
firebase_core: ^2.25.5
firebase_auth: ^4.17.6
google_sign_in: ^6.2.1
flutter_facebook_auth: ^6.1.1
google_nav_bar: ^5.0.6
cloud_firestore: ^4.15.8
text_area: ^0.0.3
google_maps_flutter: ^2.5.3
location: ^6.0.1
clipboard: ^0.1.3
firebase_database: ^10.4.9
flutter_polyline_points: ^2.0.0
image: ^4.2.0
```

Rys. 5.5 Zestawienie bibliotek oraz narzędzi użytych w projekcie znajdujących się w pliku pubspec.yaml

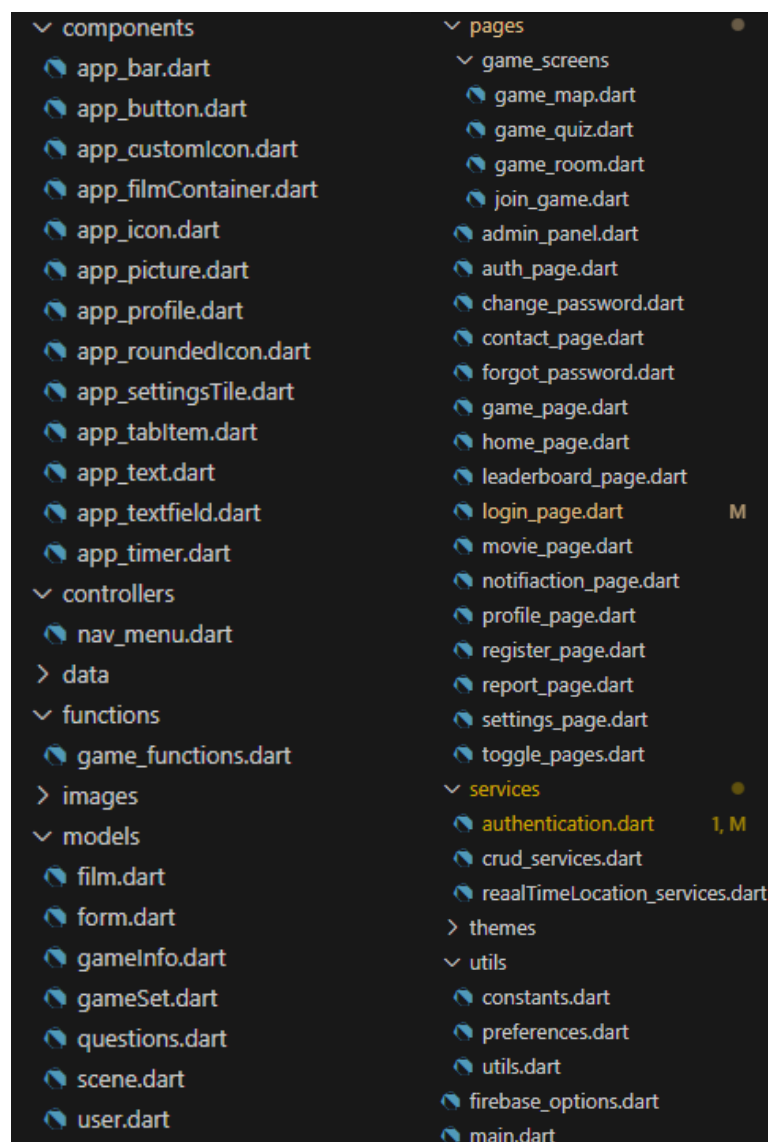
Najważniejszymi z nich są:

- **google_maps_flutter** – biblioteka odpowiadająca za włączenie map google, wymagany jest klucz API aby uzyskać dostęp do zasobów.
- **location** - niezbędna do pozyskiwania lokalizacji, zawiera serwisy oraz uprawnienia, które są pobierane od użytkownika. Pozwala również na nasłuchiwanie na zmiany lokalizacji, co jest przydatne do wyświetlania pozycji użytkownika na mapie w czasie rzeczywistym.
- **firebase_auth** – uwierzytelnianie użytkowników, możliwość rejestracji kont i zapisywanie użytkowników w sposób bezpieczny do bazy danych.

- **google_sign_in**, **flutter_facebook_auth** - umożliwia skorzystanie z zewnętrznych serwisów, aby móc zarejestrować swoje konta w aplikacji.
- **Cloud_firestore** – plugin, który daje dostęp do kolekcji utworzonych w Firebase dzięki czemu można tworzyć nowe kolekcje, aktualizować dane, wprowadzać nowe dokumenty.

5.4. Struktura projektu

Spis utworzonych plików projektowych podzielonych na odpowiednie foldery (Rys 5.6).



Rys. 5.6 Kompletna struktura plików utworzonych w projekcie

5.4.1. Modele

Modele są to klasy zbudowane pod potrzeby aplikacji, przedstawiają jakie wartości oraz typy danych są wymagane do utworzenia obiektu klasy. Obiekty są wykorzystywane w funkcjach pobierających dane z Firestore, a później używane do wypisywania informacji na ekranie np. wszystkie dane filmu (Rys. 5.7), czy informacje o danych gry użytkownika (Rys. 5.8).

```
class AppFilm{
    String uid;
    String title;
    String director;
    List screenwriters;
    String filmGenre;
    List cast;
    String year;
    double rating;
    String place;
    GeoPoint geoPoint;
    String imgPath;
    AppFilm({
        required this.uid,
        required this.title,
        required this.director,
        required this.screenwriters,
        required this.filmGenre,
        required this.cast,
        required this.year,
        required this.rating,
        required this.place,
        required this.geoPoint,
        required this.imgPath
    });
}
```

Rys. 5.7 Klasa AppFilm

```
class AppGameInfo{
    final String uid;
    final String code;
    final GeoPoint lastLocation;
    final List<String> collected;
    final String score;

    AppGameInfo({
        required this.uid,
        required this.code,
        required this.lastLocation,
        required this.collected,
        required this.score,
    });
}
```

Rys. 5.8 Klasa AppGameInfo

5.4.2. Serwisy

Logowanie (Rys. 5.9) za pomocą adresu email oraz hasła, oczekiwanie na wykonanie funkcji zagnieżdżonej w Firebase **signInWithEmailAndPassword**, która jest próbą zalogowania użytkownika w oparciu o dane autoryzujące podane podczas rejestracji. Funkcja pobiera context strony na której znajduje się użytkownik, podany adres email w formularzu oraz hasło. Context przydaje się w momencie wystąpienia błędu logowania wyświetlając odpowiedni komunikat.

```
Future<dynamic> signInWithEmailAndPassword(BuildContext context,String email, String password) async {
  try {
    final credential = await _auth.signInWithEmailAndPassword(
      email: email,
      password: password,
    );
    return null;
  } on FirebaseAuthException catch (e) {
    if(e.code=="channel-error"){
      showMessage(context,"Fields cannot be empty","Please enter your email and password corectly");
    }else {
      showMessage(context,e.code,"Incorrect email or password value");
    }
  }
}
```

Rys. 5.9 Funkcja logowania za pomocą adresu e-mail oraz hasła

SignInWithGoogle

Uwierzytelnianie z Google pozwala na logowanie użytkownika za pomocą konta tej firmy (Rys. 5.10). Wykorzystują protokół OAuth 2.0, aby uzyskać dostęp do zasobów w sposób bezpieczny. Na początku następuje utworzenie instancji **GoogleSignInAccount** oraz **GoogleSignInAuthentication**, jest to niezbędne do uruchomienia procesu wyboru konta Google przez użytkownika. Następnie pobierane są poświadczenia "credential" na podstawie tokenów po to, by móc zalogować użytkownika w bazie danych. Jeżeli jest to pierwsze użytkownika wykonywana jest również funkcja **addUser**, która tworzy nowy dokument z danymi użytkownika w Firestore.

```

Future<dynamic> signInWithGoogle( ) async {
  final GoogleSignInAccount? gUser = await GoogleSignIn().signIn();

  final GoogleSignInAuthentication gAuthentication = await gUser!.authentication;

  final gCredential = GoogleAuthProvider.credential(
    accessToken: gAuthentication.accessToken,
    idToken: gAuthentication.idToken
  );
  await _auth.signInWithCredential(gCredential);

  bool? email = await _crudServices.getUserEmail(gUser.email);
  print(email);
  if(email==false){
    _crudServices.addUser(user.uid, gUser.email, "lib/images/default-avatar.png", "0", [], "0");
  }
}

```

Rys. 5.10 Logowanie za pomocą uwierzytelniania z kontem Google

SignInWithFacebook

Funkcja oczekuje na logowanie użytkownika z Facebookiem (Rys. 5.11), następnie inicjowany jest obiekt klasy OAuthCredential, który przyjmuje poświadczenia w postaci tokenu, tak jak w przypadku logowania z Google funkcja przy pierwszym logowaniu dodaje użytkownika do kolekcji Firestore oraz posiada obsługę błędów.

```

Future<dynamic> signInWithFacebook(BuildContext context) async{
  final LoginResult result = await FacebookAuth.instance.login();

  if(result.status == LoginStatus.success){
    final OAuthCredential credential = FacebookAuthProvider.credential(result.accessToken!.token);
    final userData = await FacebookAuth.instance.getUserData();
    try{
      await _auth.signInWithCredential(credential);
      bool? email = await _crudServices.getUserEmail(userData['email']);
      print(email);
      if(email==false){
        _crudServices.addUser(user.uid, userData['email'], "lib/images/default-avatar.png", "0", [], "0");
      }
    }on FirebaseAuthException catch (e){
      return showMessage(context, e.code, "Can't authorize your facebook account");
    }
  }else{
    print(result.status);
    print(result.message);
  }
}

```

Rys. 5.11 Logowanie za pomocą uwierzytelniania z kontem Facebook

SignUpWithEmailAndPassword

Rejestracja użytkownika odbywa się za pomocą funkcji dynamicznej przyjmujące wartości z formularza tj. adres e-mail, hasło, potwierdzenie hasła (Rys. 5.12). Następuje wykonanie warunków które w przypadku, gdy hasło nie równa się potwierdzonemu hasłu to zostaje wyświetlony specjalny komunikat. Gdy warunek zostanie spełniony, zostaje

wykonana wbudowana funkcja Firebase **createUserWithEmailAndPassword** oraz użytkownik zostaje dodany do kolekcji Firestore.

```
Future<dynamic> signUpWithEmailAndPassword(BuildContext context,String email, String password, String confirm) async{
  try{
    if(password==confirm){
      await _auth.createUserWithEmailAndPassword(email: email, password: password);

      _crudServices.addUser(
        user!.uid,
        email,
        "https://e7.pnggg.com/pngimages/647/460/png-clipart-computer-icons-open-person-family-icon-black-silhouette-black.png",
        "0",
        [],
        "0"
      );
    }else{
      return null;
    }
  } on FirebaseAuthException catch (e){
    if(e.code=="channel-error"){
      showMessage(context,"Fields cannot be empty","Fill all the registration inputs");
    }else if(e.code=="invalid-email"){
      showMessage(context, e.code, "Enter your email address correctly");
    }
    else if(e.code=="email-already-in-use"){
      showMessage(context,e.code,"Account exists");
    }else{
      showMessage(context,e.code,"Password must be at least 6 characters");
    }
  }
}
```

Rys. 5.12 Rejestracja użytkownika z wykorzystaniem adresu e-mail oraz hasła

DeleteAccount

Pobranie numeru identyfikującego użytkownika przez funkcję **getUserIdByUid** jest konieczne, aby zwrócić wartość id dokumentu z Firestore w jakim znajdują się jego dane. Jeśli wartość zalogowanego użytkownika nie jest pusta, to z kolekcji usuwany jest dokument oraz usuwany jest użytkownik z Firebase (Rys. 5.13). Po usunięciu konta częstym błędem było pozostanie użytkownika zalogowanego na koncie, które nie istnieje, zapobiegnięciem wystąpienia tego błędu było dodanie wykonania metody **signOut()** po wszystkich operacjach.

```

deleteAccount()async{
  String? uid = await _crudServices.getUserIdByUid(_auth.currentUser!.uid.toString());
  try{
    User? user= _auth.currentUser;

    if(user != null){
      await FirebaseFirestore.instance
        .collection('users')
        .doc(uid)
        .delete();
      await user.delete();
      FirebaseAuth.instance.signOut();
      print("Account deleted");
    }else{
      print("Couldn't find current user");
    }
  }catch (e){
    print("error: $e");
  }
}

```

Rys. 5.13 Funkcja usuwająca konto użytkownika za bazy danych

AddUser

Dane użytkownika są przyjmowane jako parametry funkcji, na podstawie których tworzona jest mapa, gdzie każdy klucz znajdujący się w Firestore otrzymuje nowe wartości. Następnie za pomocą metody wywoływana jest próba dodania nowego dokumentu wraz z polami do bazy danych (Rys. 5.14).

```

Future addUser(String uid, String email, String imagePath, String score, List visited, String gamesPlayed) async{
  Map<String,dynamic> userData = {
    "uid":uid,
    "email":email,
    "imagePath":imagePath,
    "score":score,
    "visited":visited,
    "gamesPlayed":gamesPlayed,
    "isAdmin":false
  };

  try{
    await _firestore.collection("users").add(userData);
    print("Data added");
  }catch(e){
    print(e.toString());
  }
}

```

Rys. 5.14 Dodanie użytkownika do kolekcji users

GetUserIdByUid

W przypadku próby uzyskania dostępu do konkretnego dokumentu w bazie danych jest potrzebny jego klucz id. Poniższa funkcja (Rys. 5.15) pobiera wszystkie dokumenty z Firestore w których wystąpiło id użytkownika (argument wejściowy funkcji). Jeśli wartość z wykonanego zapytania nie jest pusta to funkcja zwraca pierwszy znaleziony klucz. W przeciwnym wypadku zostanie zwrócona wartość pusta.

```
Future<String?> getUserIdByUid(String currentUserId) async {
  try {
    QuerySnapshot<Map<String, dynamic>> querySnapshot =
      await FirebaseFirestore.instance.collection('users').where('uid', isEqualTo: currentUserId).get();
    if (querySnapshot.docs.isNotEmpty) {
      return querySnapshot.docs.first.id;
    } else {
      return null;
    }
  } catch (e) {
    print('Error getting user ID by UID: $e');
    return null;
  }
}
```

Rys. 5.15 Funkcja zwracająca id dokumentu jako wartość tekstową

SetAdminState

Metoda, która przyjmuje jako argument id użytkownika, za pomocą funkcji **getUserIdByUid** pobierany jest numer identyfikujący dokument. Następnie wykonywana jest aktualizacja pola **isAdmin** dla wcześniej przygotowanej referencji (Rys. 5.16), czyli odniesienia się do dokumentu użytkownika, w którym znajdują się jego dane.

```
Future setAdminState(String uid) async {
  String? userDocId = await getUserIdByUid(uid);
  final ref = _firestore.collection('users').doc(userDocId);
  ref.update({
    "isAdmin": true,
  });
  print("data updated");
}
```

Rys. 5.16 Funkcja aktualizująca uprawnienia użytkownika

GetUsersRanking

Klasa Stream pozwala na odbieranie sekwencji zdarzeń, aby przedstawić ranking niezbędne jest posortowanie wyników punktowych od największych do najmniejszych z kolekcji użytkowników, a następnie zwrócenie wartości zapytania (Rys. 5.17).

```
Stream<QuerySnapshot> getUsersRanking() async*{
  final usersQuery = _firestore.collection("users").orderBy("score",descending: true).snapshots();
  yield* usersQuery;
}
```

Rys. 5.17 Funkcja zwracająca wynik zapytania jakim jest posortowany ranking użytkowników

5.4.3. Kontrolery

Nav Controller

Kontroler menu, który zmienia zawartości ekranu w zależności od wybranej zakładki. W opcjach widżetu (Rys. 5.18) znajdują się klasy reprezentujące różne strony aplikacji, każda opcja jest osobnym indeksem.

```
List<Widget> _widgetOptions = <Widget>[
  HomePage(),
  ProfilePage(),
  GamePage(),
  Leaderboard(),
  SettingsPage()
];
```

Rys. 5.18 Lista klas w których znajdują się zawartości stron z menu dolnego

W zależności od wybranej opcji, zmienia się indeks strony (Rys. 5.19), która ma zostać wyświetlona dla użytkownika, w tym celu pobierana jest klasa z listy o określonym indeksie.

```
body: Center(
  child: _widgetOptions.elementAt(_selectedIndex)
), // Center
```

Rys. 5.19 Przypisywanie elementu pochodnego poprzez wybrany indeks

Atrybut **bottomNavigationBar** (Rys. 5.20) jest zagnieżdżony w strukturze strony, który umożliwia tworzenie pasków nawigacyjnych. Aby przedstawić w sposób nowoczesny menu oraz elementy w nim zawarte używana jest zewnętrzna biblioteka **GNav**. Cechy kolorystyczne oraz dotyczące rozmiaru ustawiane są w atrybutach tj. np. Kolor efektów, odstępy, wielkość ikon, kolor tła. Atrybut tabs zawiera przyciski odnoszące się do stron, które mają zostać wyświetlone po naciśnięciu.

```

bottomNavigationBar: Container(
  color: Colors.black,
  child: Padding(
    padding: const EdgeInsets.symmetric(horizontal: 8.0, vertical: 8.0),
    child: GNav(
      rippleColor: Colors.grey[800]!,
      hoverColor: Colors.grey[700]!,
      gap: 8,
      activeColor: Colors.white,
      iconSize: 24,
      padding: EdgeInsets.all(20),
      duration: Duration(milliseconds: 400),
      tabBackgroundColor: Colors.grey[900]!,
      color: Colors.white,
      backgroundColor: Colors.black,

      tabs: [
        GButton(icon: Icons.map, text: 'Home',),
        GButton(icon: Icons.person, text: 'Profile',),
        GButton(icon: Icons.gamepad, text: 'Play',),
        GButton(icon: Icons.leaderboard, text: 'Ranking',),
        GButton(icon: Icons.settings, text: 'Settings',),
      ],
    ),
  ),
)

```

Rys. 5.20 Atrybut klasy BottomNavigationBar wraz z formatowaniem

5.4.4. Komponenty

Tworzenie komponentów usprawnia wprowadzanie gotowych bloków konstrukcyjnych na stronę. Niestanowe widżety nie zmieniają się po zbudowaniu strony, co oznacza, że są wykorzystywane podczas wyświetlania danych statycznych. Stanowe widżety mogą zmieniać się pod wpływem działań użytkownika.

AppButton

Przykładowym komponentem jest np. klasa przycisków, które znajdują się w różnych częściach projektu (Rys. 5.21), widżet przyjmuje wartości tytułu na przycisku w formie tekstowej oraz funkcję, która ma się wykonać po jego naciśnięciu.

```

Widget build(BuildContext context) {
  return Container(
    padding: EdgeInsets.all(10),
    margin: EdgeInsets.symmetric(horizontal: 25),
    decoration: BoxDecoration(
      color: Colors.black,
      borderRadius: BorderRadius.circular(5)
    ), // BoxDecoration
    child: TextButton(
      onPressed: onPressed,
      child: Center(
        child: Text(
          buttonText,
          style: TextStyle(fontWeight: FontWeight.bold, color: Colors.white, fontSize: 16),
        ), // Text
      ), // Center
    ), // TextButton
  ); // Container
}

```

Rys. 5.21 Stylizacja komponentu przycisków

AppIcon

Komponent, który reaguje na gesty, w tym przypadku jest to kliknięcie przez użytkownika na ikonę. Klasa przyjmuje wartości funkcji jaka zostanie wykonana po kliknięciu oraz ścieżkę do pliku z konkretnym zdjęciem. Reszta atrybutów przeznaczona jest do dekoracji otoczenia ikony np. ustawienie okrągłej ramki i jej koloru (Rys. 5.22).

```
Widget build(BuildContext context) {  
  return GestureDetector(  
    onTap: onTap,  
    child: Container(  
      padding: const EdgeInsets.all(15),  
      margin: EdgeInsets.symmetric(horizontal: 10),  
      decoration: BoxDecoration(  
        border: Border.all(color: Colors.black12),  
        borderRadius: BorderRadius.circular(16),  
        color: Colors.black12,  
      ), // BoxDecoration  
      child: Image.asset(imagePath,height: 50,),  
    ), // Container  
  ); // GestureDetector  
}
```

Rys. 5.22 Stylizacja komponentu ikon

AppTextField

Jest to klasa komponentu przedstawiającego pole tekstowe (Rys. 5.23) dla pojedynczej frazy np. dla wprowadzania wartości hasła czy adresu e-mail w formularzu. Widżet poprawnie skonstruowany potrzebuje wartości tj.:

- **Controller** – kontroler za pomocą którego można pobrać wartości podane przez użytkownika w formularzu.
- **IfObscureText** – jest to zmienna bool, która w zależności, od przyjętej wartości ukrywa zawartość wpisanej treści przez użytkownika lub pokazuje.
- **HintText** – odpowiedź, która sugeruje do czego przeznaczone jest pole tekstowe.

```

Widget build(BuildContext context) {
  return Padding(
    padding: const EdgeInsets.symmetric(horizontal: 25.0),
    child: TextField(
      controller: controller,
      obscureText: ifObscureText,
      decoration: InputDecoration(
        enabledBorder: const OutlineInputBorder(
          borderSide: BorderSide(color: Colors.grey, width: 2),
        ), // OutlineInputBorder
        focusedBorder: const OutlineInputBorder(
          borderSide: BorderSide(color: Colors.black, width: 2),
        ), // OutlineInputBorder
        fillColor: Colors.white,
        filled: true,
        hintText: hintText,
        hintStyle: TextStyle(color: Colors.grey[500])
      ), // InputDecoration
    ), // TextField
  ); // Padding
}

```

Rys. 5.23 Stylizacja komponentu pola tekstowego

AppFilmContainer

Dla konkretnego numeru id filmu widżet przypisuje obiekt klasy **AppFilm** (Rys. 5.24). Dzięki klasie **FutureBuilder** dane filmu są pobierane z bazy danych w sposób asynchroniczny na stronie, a później przedstawione w stylizowanym kontenerze. W przypadku wystąpienia błędu na stronie wyświetlany jest error oraz w jeśli dane wymagają dłuższego czasu wyświetlania, to w miejscu kontenera za pomocą klasy **CircularProgressIndicator** wyświetlany jest obiekt ładowania.

```

Widget build(BuildContext context) {
  Future<AppFilm?> _preferences = Preferences().getfilmInfo(filmUid);
  return Container(
    height: 40.0,
    width: 50.0,
    child: FutureBuilder<AppFilm?>({
      future: _preferences,
      builder: (context, snapshot){
        if(snapshot.connectionState==ConnectionState.waiting){
          return Center(child: CircularProgressIndicator(),);
        }else if(snapshot.hasError){
          return Center(child: Text("Error: ${snapshot.error}"));
        }else{
          AppFilm? film = snapshot.data;
          return GestureDetector(
            onTap: () {
              Navigator.push(context, MaterialPageRoute(builder: (context) =>
                MoviePage(
                  movie: AppFilm(

```

Rys. 5.24 Budowa komponentu kontenera wywoływanego na konkretnej wartości z bazy danych

5.4.5. Funkcje

Metody pomocnicze ułatwiające pobieranie, aktualizowanie danych na różnych stronach. Dane są zaciągane z bazy danych dla konkretnych wartości wywołujących funkcje np. Numer identyfikujący pozwalający na pozyskanie wartości odnoszących się do pożądanego dokumentu z Firestore.

GetCurrentUserInfo

Funkcja działająca w sposób asynchroniczny dla aktualnie zalogowanego użytkownika pobierająca zawartość jego dokumentu z kolekcji użytkowników (Rys. 5.25). **DocumentSnapshot** znajduje dokument i dane w nim zawarte dla podanego id. Jeżeli wartość obiektu nie jest pusta to dane są mapowane do obiektu **AppUser** oraz zwracane. Lista “visit” jest listą pomocniczą, ponieważ gdyby zmapować od razu dane do pola visited, zostałby automatycznie wybrany **Map<Object?, Object?>** jako typ danych, co uniemożliwiałoby wykonywanie operacji na danych z listy.

```
Future<AppUser?> getCurrentUserInfo() async{
  try{
    String? userId = await CrudServices().getUserIdByUid(_auth.currentUser!.uid);
    print(userId);
    DocumentSnapshot querySnapshot =
      await _firestore.collection("users").doc(userId.toString()).get();

    if(querySnapshot.exists){
      Map<String, dynamic>? userData = querySnapshot.data() as Map<String,dynamic>;
      List<String> visit = List<String>.from(userData['visited']);
      return AppUser(
        imagePath: userData['imagePath'],
        email: userData['email'],
        score: userData['score'],
        visited: visit,
        gamesPlayed: userData['gamesPlayed']
      );
    }else{
      return null;
    }
  }catch (e){
    print(e.toString());
    return null;
  }
}
```

Rys. 5.25 Funkcja zwracająca obiekt *AppUser*

AddToCollection

Aby dodać do kolekcji miejsce, które zostało zebrane przez użytkownika spełniającego warunki odległości, funkcja (Rys. 5.26) musi pobrać dokument przypisany do id użytkownika z bazy danych. Następnie w miejscu pola “visited” wykonywana jest aktualizacja, stosując referencje do tego pola. Lista, która jest wartością klucza “visited” zostaje powiększona dzięki łączeniu tablic o kolejną wartość dokumentu odnoszącego się do konkretnego filmu w kolekcji.

```
Future addToCollection(String docId)async{
  String? userDocId = await _crudServices.getUserIdByUid(_auth.currentUser!.uid);
  final ref = _firestore.collection('users').doc(userDocId);
  ref.update({
    "visited": FieldValue.arrayUnion([docId]),
  });
  print("data updated");
}
```

Rys. 5.26 Funkcja aktualizująca pole visited w bazie danych dla kolekcji users

5.4.6. Strony

Strona logowania oraz rejestracji zbudowana jest z kilku klas komponentów oddzielonych od siebie marginesami. Między innymi stosowany jest **AppPicture** (Rys. 5.27), czyli komponent tworzący ramkę do zdjęcia, którego ścieżka podana jest w atrybucie oraz atrybutem jest wielkość wyświetlanego kontenera.

```
SizedBox(height: 50),
AppPicture(imagePath: 'lib/images/main-logo.png' , size: 170),
SizedBox(height: 20),
```

Rys. 5.27 Funkcja aktualizująca pole visited w bazie danych dla kolekcji users

Na stronie występuje również **GestureDectector** (Rys. 5.28), który pozwala na wybranym widżecie ustawić funkcję jaka powinna się wykonać po kliknięciu. W tym przypadku stosowany jest do nawigowanie na stronę, która pozwala na zresetowanie hasła.

```
GestureDetector(
  onTap: (){
    Navigator.pushNamed(context, '/forgot_password');
  },
  child: Text('Forgot password?', style: TextStyle(color: Colors.blue, fontWeight: FontWeight.bold)),
) // GestureDetector
```

Rys. 5.28 Klasa GestureDectectior wraz z stylizacją

W aplikacji występują strony, które przedstawiają zawartość różnych kolekcji z bazy danych. Głównym elementem rozpoczynającym implementację jest pobranie zmapowanego obiektu klasy z kolekcji tak, aby w łatwy sposób odnosić się do zawartości. Preferencje dotyczące pobieranych danych są określane za pomocą specjalnej klasy Preferences, której użycie wygląda podobnie do przedstawionej wcześniej funkcji (Rys. 5.24), gdzie wartości obiektu są mapowane z bazy danych.

Stosowanie **FutureBuilder** (Rys. 5.29) na obiekcie jest potrzebne do obserwowania zmian zachodzących w bazie danych. Użycie atrybutu “builder” pozwala na używanie zapytań w sposób asynchroniczny, dzięki czemu można wielokrotnie zmieniać stan danych na stronie. W przypadku braku danych zostanie wyświetlony error na lub w niektórych przypadkach pusty kontener.

Struktura stron opiera się na kontenerach, łatwo można je dostosować stosując statyczne wartości co do wielkości elementu, czy ustawić kolorystyczne preferencje w dekoracjach elementu. Na stronie wykorzystywane są również klasy komponentów **AppText** (Rys. 5.29), aby w łatwy sposób dostosowywać tekst.

```
Container(
  width: 100,
  child: Column(
    // ignore: prefer_const_literals_to_create_immutables
    children: [
      AppText().titleText(user?.score ?? "0", Colors.blue),
      AppText().subtitleText("Your Score", Colors.grey[500]!),
    ],
  ), // Column
), // Container
```

Rys. 5.29 Klasa wraz z metodami komponentów tytułowych

Klasą używaną głównie w profilu użytkownika jest **GridView** (Rys. 5.30), który pozwala na przedstawienie danych w tabeli. Można dostosowywać ilość elementów jakie mają być wyświetlane w wierszu oraz jakie to będą elementy.

```
if(user?.visited !=null)
  GridView.count(
    crossAxisCount: 2,
    shrinkWrap: true,
    childAspectRatio: 1/1.5,
    children: [
      for(int i=0;i<=user!.visited.length-1;i++)
        Padding(
          padding: const EdgeInsets.all(8.0),
          child: FilmContainer(filmUId: user.visited[i]),
        ) // Padding
    ],
  ) // GridView.count
```

Rys. 5.30 Klasa przedstawiania danych w tabeli

Visibility jest klasą sprawdzającą czy postawiony warunek, który jest atrybutem koniecznym został spełniony. W przypadku użytej funkcji **nerbyLocation** zostaje sprawdzona odległość między lokalizacją użytkownika, a lokalizacją docelową. Kiedy warunek zostanie spełniony to zostanie wyświetlony przycisk, za pomocą którego do kolekcji użytkownika dodawany jest numer id filmu, którego miejsce nagrywania właśnie odwiedził.

```
Visibility(
  visible: nerbyLocation(LatLng(movie.geoPoint.latitude, movie.geoPoint.longitude), currentPosition)&&inGame==false,
  child: AppButton(buttonText: "Add to collection", onPressed: (){Utils().addToCollection(movie.uid);Navigator.pop(context);}),
), // Visibility
```

Rys. 5.31 Klasa **Visibility** wyświetlająca lub ukrywająca element na stronie

5.4.7. Wzór Haversine

Do wyznaczania odległości między lokalizacjami został użyty wzór **Haversine** (ang. *Half of the sinus versus*). Obliczanie polega na zmierzeniu odległości między dwoma punktami na kuli ziemskiej na podstawie długości i szerokości geograficznej. Na początku należy wyznaczyć centralny kąt między dwoma punktami na kuli ziemskiej używając wzoru:

$$\Theta = \frac{d}{r}$$

gdzie:

d – dystans między dwoma punktami wzdłuż wielkiego koła,

r – promień kuli.

Zastosowanie formuły pozwala na obliczenie dystansu bezpośrednio używając długości i szerokości geograficznej [16]:

$$d = 2r \arcsin \left(\sqrt{\sin^2 \left(\frac{\varphi_2 - \varphi_1}{2} \right) + \cos(\varphi_1) \cos(\varphi_2) \sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right)} \right)$$

gdzie:

φ_1, φ_2 – punkty szerokości dwóch lokalizacji,

λ_1, λ_2 – punkty długości dwóch lokalizacji,

$\Delta\varphi = \varphi_2 - \varphi_1$ – różnica szerokości geograficznej.

$\Delta\lambda = \lambda_2 - \lambda_1$ – różnica długości geograficznej.

Do obliczenia dystansu została napisana funkcja, pierwszą czynnością była zamiana stopni na radiany, czyli przemnożenie szerokości i długości geograficznej przez $\left(\frac{\pi}{180}\right)$.

Następnie zostały wykonane kolejne obliczenia poprzez podstawienie już gotowego wzoru haversine [19]:

$$a = \sin^2 \left(\frac{\varphi_2 - \varphi_1}{2} \right) + \cos(\varphi_1) \cos(\varphi_2) \sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right)$$

$$c = 2 \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{1-a})$$

$$d = R \cdot c$$

gdzie:

R – stała określająca promień ziemi, wynoszący średnio 6371 km.

Otrzymany wynik odległości będzie w kilometrach, dlatego trzeba go pomnożyć przez 1000, aby w łatwy sposób sprawdzać odległość między użytkownikiem, a miejscem

docelowym. Poniżej została przedstawiona implementacja funkcji (Rys. 5.32) przyjmująca 2 stałe, liczbę π oraz promień ziemi.

```
double getDistance(LatLng cLocation, LatLng dLocation){
    const double pi = 3.1415926535897932;
    const double radius = 6371;
    double lat1 = cLocation.latitude*(pi/180);
    double long1 = cLocation.longitude*(pi/180);
    double lat2 = dLocation.latitude*(pi/180);
    double long2 = dLocation.longitude*(pi/180);
    double latDiff = lat2-lat1;
    double longDiff = long2-long1;
    double a = sin(latDiff/2) * sin(latDiff/2) + cos(lat1) * cos(lat2) * sin(longDiff/2) * sin(longDiff/2);
    double c = 2 * atan2(sqrt(a), sqrt(1-a));
    double d = radius * c * 1000;
    print("distance beetwen current location and destination:${d}");
    return d;
}
```

Rys. 5.32 Funkcja stosująca wzór Haversine do obliczania odległości między punktami na ziemi

5.5. Gra terenowa

Rozpoczęcie gry zaczyna się na stronie głównej (Rys. 5.33), gdzie użytkownik posiada dwie drogi. Wybranie przycisku “**CreateGame**” automatycznie przeniesie użytkownika do pokoju, skąd będzie mógł przekazać kod gry drugiemu użytkownikowi. Jeśli wybierze przycisk “**Join game**” zostanie przeniesiony do strony i będzie musiał wprowadzić odpowiedni kod gry.

```
AppButton(
  buttonText: "Create game",
  onPressed: (){
    GameFunctions().createGame(context);
  }
), // AppButton
SizeBoxSizeBox(height: 20,),
AppButton(
  buttonText: "Join game",
  onPressed: (){
    Navigator.pushNamed(context, '/join_game');
  }
), // AppButton
```

Rys. 5.33 Klasa *GamePage* wyświetlająca możliwości w jakie użytkownik może dołączyć do rozgrywki

Utworzenie rozgrywki przez użytkownika

Funkcja **createGame** (Rys. 5.34) tworzy nową referencję do Realtime Database, wprowadzając wartość id użytkownika tworzącego pokój. Następnie użytkownik jest przenoszony strony **GameRoom**, czyli pokoju, którego kod jest wcześniej generowany przez funkcję **generateCode** (Rys. 5.35).

```

Future<void> createGame(BuildContext context) async {
  DatabaseReference roomsRef = database.child('/rooms');
  final String gameCode = generateCode();
  try{
    final newRoom = <String, dynamic>{
      'player1_uid': user!.uid,
      'player2_uid': "",
    };
    roomsRef.child(gameCode).set(newRoom)
      .then((value) =>
        print('Document ID for the new data: $gameCode'))
      .catchError(
        (error) => print('${error}')
      );
    print('Game ID: ${gameCode}');
    Navigator.push(context, MaterialPageRoute(builder: (context) => GameRoomScreen(
      gameId: gameCode,
    ))); // GameRoomScreen // MaterialPageRoute
  } on Exception catch (e) {
    print("$e");
  }
}

```

Rys. 5.34 Funkcja tworząca referencje do bazy danych i nawigująca do kolejnej strony

Do wygenerowania kodu funkcja sprawdza w pętli czy podany kod nie jest przypisany do innego pokoju albo nie jest pusty. Spełnienie warunku powoduje wykonanie algorytmu, w którym wykorzystuje się znaki klawiaturowe bez znaków polskich do losowania randomowego ciągu znaków składającego się z 6 liter lub liczb. Następnie taki kod jest zwracany i przypisywany do pokoju. W tym przypadku została zastosowana klasa **Random**, czyli generatora liczb, znaków, wartości bool pseudolosowych.

```

String generateCode(){
  String code = '';
  while(codeExists(code)==true||code==''){
    code=buildCode();
  }
  return code;
}
String buildCode(){
  const String chars = "QWERTYUIOPASDFGHJKLZXCVBNM1234567890";
  final Random random = Random();
  String code = '';
  int index;
  for(int i=0;i<6;i++){
    index=random.nextInt(chars.length);
    code+= chars[index];
  }
  return code;
}

```

Rys. 5.35 Funkcja generująca losowy ciąg cyfr oraz liter składający się na 6 znakowy kod

Dołączenie użytkownika do istniejącej rozgrywki.

Dla użytkownika, który dołącza do rozgrywki wyświetlona zostaje strona z możliwością podania kodu w polu tekstowym. Następnie po kliknięciu w przycisk

wykonywana jest funkcja **joinGame** (Rys 5.36) przyjmująca jako argumenty kontekst oraz wartość tekstową przechowywaną przez kontroler.

Funkcja **joinGame** pobiera pojedynczą odpowiedź na zdarzenie za pomocą **DatabaseEvent** z bazy danych, która zawiera informacje o zmianach jakie zaszły. Używając klasy **DataSnapshot** odczytywane są dane, które zostały pobrane z konkretnego węzła drzewa JSON-owego. Jest to sprawdzenie, czy w bazie danych został utworzony pokój o podanym kodzie przez użytkownika. Jeśli warunek zostanie spełniony, czyli jeśli wartość pobranych danych nie jest pusta, to dane użytkownika zostają zapisane do bazy, a następnie zostaje on przeniesiony do pokoju, gdzie jest w stanie odczytywać dane swojego przeciwnika. W przypadku podania błędnego kodu lub jeśli już pokój o danym kodzie nie istnieje w bazie danych, zostanie wyświetlony komunikat o błędzie.

```
Future<void> joinGame(BuildContext context, String code)async{
  final String userId = user!.uid;
  final DatabaseEvent event = await database.child('/rooms').child(code).once();
  final DataSnapshot snapshot = event.snapshot;
  final value = <String, dynamic>{
    'player2_uid': userId,
  };
  if(snapshot.exists){
    try{
      database.child('/rooms').child(code).update(value).then(
        (value) => print("User ${userId} is joining the game")
      ).catchError(
        (error) => print("${error}")
      );
      Navigator.push(context, MaterialPageRoute(builder: (context) => GameRoomScreen(
        gameId: code,
      ))); // GameRoomScreen // MaterialPageRoute
    }on Exception catch (e) {
      print("$e");
      showMessage(context, "$e", "error");
    }
  }else{
    showMessage(context, "Room doesn't exists!", "type correct id again");
  }
}
```

Rys. 5.36 Funkcja odpowiadająca za dodawanie referencji użytkownika do bazy danych oraz wyświetlenie strony pokoju

GameRoom

Klasa nasłuchuje na zdarzenie zmian występujących na dziecku węzła “rooms” (Rys. 5.37). Przechowywane są tam numery id użytkowników, jeżeli któraś z wartości ulegnie zmianie, zostaje wykonana funkcja, która pobiera zawartość ścieżki. Dane są przechowywane w mapie i w momencie aktualizacji stanu przypisywane do zmiennych. Jeśli dane są puste co może być spowodowane wyjściem osoby hostującej z pokoju to użytkownik, który dołączył do rozgrywki zostaje z niej wyrzucony, nawigator przenosi go warstwę wcześniej, czyli do ekranu dołączania do rozgrywki.

```

_roomSubStream=database.child('/rooms').child(widget.gameId).onValue.listen((event) {
  Object? dataSnapshot = event.snapshot.value;
  if (dataSnapshot != {}) {
    if (dataSnapshot is Map<Object?, Object?>) {
      Map<Object?, Object> mapData = Map<Object?, Object>.from(dataSnapshot);

      print('Data retrieved successfully: $mapData');
      if(mounted){
        setState(() {
          host= mapData['player1_uid'] as String? ?? "";
          player2 = mapData['player2_uid'] as String? ?? "";
        });
      }
    } else {
      print('Error: Unexpected data type: ${dataSnapshot.runtimeType}');
    }
  } else {
    if(user!.uid==player2){
      setState(() {
        Navigator.pop(context);
      });
    }else{
      print('Error: Data is null');
    }
  }
}

```

Rys. 5.37 Nasłuchiwanie na zmiany w referencjach na węźle rooms o podanym kodzie gry

Kolejne nasłuchiwanie odbywa się na dziecku węzła “users” (Rys. 5.38), dane w podobny sposób przechowywane są jako mapa oraz przypisywana jest wartość bool gotowości do innego obiektu mapy. Jeśli mapa nie jest pusta oraz wartość **readyValue** dla obu użytkowników jest prawdziwa, nawigator przenosi obydwójce użytkowników do mapy rozgrywki przyjmując ich numery id.

```

_readyStateStream=database.child('/users').child(host).onValue.listen((event) {
  if(event.snapshot.value!=null){
    Object? dataSnapshot = event.snapshot.value;
    print("snapshot: ${dataSnapshot.toString()}");
    if (dataSnapshot != {}) {
      if (dataSnapshot is Map) {
        Map<Object?,Object?> mapData = dataSnapshot as Map<Object?,Object?>;
        Map<Object?,Object?> readyValue = mapData[host] as Map<Object?,Object?>;
        print("ready value:${readyValue}");
        if(mounted){
          setState(() {
            print("mapdata: ${readyValue['ready']}");
            p1=readyValue['ready'] as bool? ?? false;
            print(p1);
            if(p1==true&&p2==true){
              Navigator.push(context, MaterialPageRoute(builder: (context) => GameMap(gameId: widget.gameId,user1: host,user2: player2,)));
            }
          });
        }
      }
    }
  }else{
    print("data is null");
  }
}
else{
  print("data is null");
}
}

```

Rys. 5.38 Nasłuchiwanie na zmiany w referencjach na węźle users dla pobranego id użytkownika oraz sprawdzanie gotowości

Przy pierwszym dołączeniu do pokoju, w procesie inicjacji stanu widżetu wywoływana jest funkcja **createRoomHistory** (Rys. 5.39). Jako pierwsze wykonywane jest sprawdzenie czy funkcja **roomExists**, która odpowiedzialna jest za zwrócenie wartości prawdziwej w przypadku, jeśli pokój o danym kodzie istnieje w kolekcji Firestore. W przeciwnym wypadku wywoływana jest funkcja **addGameRoom**. Kolejnym zadaniem funkcji jest pobranie listy wszystkich zestawów gier z kolekcji “gameSets” z Firestore, a następnie wybranie losowego obiektu **AppGameSet** za pomocą klasy **Random**. Gdy zestaw gry został ustawiony to jego numer identyfikujący zapisywany jest w dokumencie odnoszącym się do historii pokoju wywołując funkcję **setGameSet**. Funkcje wypisane w tym akapicie są zgodne z zasadami modyfikującymi kolekcje Firestore opisane w podrozdziale (5.4.5).

```
Future<void> _createRoomHistory() async{
  bool exists=await _functions.roomExists(user!.uid, widget.gameId);
  if(mounted){
    setState(() {
      if(exists==false){
        _functions.addGameRoom(
          AppGameInfo(
            uid: user!.uid,
            code: widget.gameId,
            lastLocation: GeoPoint(0, 0),
            collected: List.empty(),
            score: "0"
          ) // AppGameInfo
        );
      }
    });
    final Random random = new Random();
    List<AppGameSet> gameSets = await _functions.getRandomGameSet();
    AppGameSet gameSet = gameSets[random.nextInt(gameSets.length)];
    setState(() {
      _functions.setGameSet(widget.gameId, gameSet.uid);
    });
  }
}
```

Rys. 5.39 Funkcja dodająca informacje o utworzonym pokoju do bazy danych w kolekcji rooms

Funkcja **onBackChanges** (Rys. 5.40) jest wywoływana w momencie powrotu do strony głównej z ekranu pokoju. Tworzona jest lista użytkowników do której są przypisywane numery id, jeśli użytkownik znajdował się w pokoju. Za pomocą funkcji **removeUserData** usuwane są dane przechowywane w czasie rzeczywistym w węźle “users”. **DeleteRoomHistory** wywołuje usunięcie z kolekcji dokumentów, w których znajdowały się numery id użytkowników oraz kod gry.

```

void _onBackChanges(){
    List<String> users = List.empty(growable: true);
    if(user!.uid == host){
        users.add(host);
        final removeRef = database.child('/rooms').child(widget.gameId);
        _removeUserData(user!.uid);
        if(player2!=""){
            _removeUserData(player2);
            users.add(player2);
        }
        _functions.deleteRoomHistory(users, widget.gameId);
        removeRef.remove();
    }
}

```

Rys. 5.40 Usunięcie historii użytkowników oraz wszystkich elementów odnoszących się do pokoju rozgrywki

Jeżeli użytkownik hostujący znajduje się w pokoju to zostaje wyświetlony kontener, którego zawartość jest wypisywana, aktualizowana za pomocą klasy **FutureBuilder**.

Jako atrybut “future” przyjmowana jest funkcja **getUserInfo** mapująca zawartość kolekcji użytkownika, co pozwoli aktualizować na bieżąco zmiany zachodzące na jego koncie. Zwracany jest pojedynczy element listy dla każdego z użytkowników oraz ustawiona jest klasa **Visibility** sprawdzająca czy użytkownik zatwierdził swoją gotowość, wtedy wyświetlana jest ikona przy jego danych.

GameMap

Jest to strona używająca **Google Maps API**, rozpoczyna rozgrywkę inicjując takie funkcje jak np. zegar, który odlicza czas czy nasłuchiwanie na zmiany lokalizacji użytkownika. Funkcją zarządzającą zmianami lokalizacji jest **onLocationChanges** (Rys. 5.41), rozpoczyna od zainicjowania dwóch zmiennych tj. sprawdzająca czy użytkownik włączył usługę lokalizacji, czy aplikacja posiada uprawnienia do lokalizacji. Następuje weryfikacja usługi lokalizacyjnej, jeśli jest wyłączona to kończy działanie i wraca użytkownika do ekranu głównego. Zmienna **premissionGranted** oczekuje na zatwierdzenie uprawnień, jeśli takowe zostaną odrzucone to kończone jest działanie. Kiedy uprawnienia i usługi są potwierdzone, włączone, tworzona jest subskrypcja na zmianach lokalizacji jest to niezbędne do kontrolowania strumienia danych. **StreamSubscription** to klasa, która pozwala na wstrzymywanie, opóźnianie i anulowanie subskrypcji na danym zdarzeniu, ważne jest, aby w przypadku opuszczania strony klasa zatrzymywała nasłuchiwanie na strumieniu danych. Właściwość **onLocationChanged** pozwala, aby kontroler mógł nasłuchiwać na aktualną lokalizację użytkownika. Następuje sprawdzenie czy wartości długości i szerokości geograficznej nie są puste, gdy warunek zostanie spełniony dodatkowym sprawdzeniem jest, czy wartości lokalizacji są takie same jak już wcześniej ustawiona lokalizacja. Jest to optymalne zachowanie w przypadku, gdy użytkownik nie zmienia swojego położenia na mapie, wtedy żadne inne funkcje nie są wykonywane tj. aktualizacja znacznika użytkownika czy zapis lokalizacji użytkownika do bazy danych.

```

void _onLocationChanges() async{
  bool _serviceEnabled;
  PermissionStatus _permissionGranted;

  _serviceEnabled = await _locationController.serviceEnabled();
  if(_serviceEnabled){
    _serviceEnabled = await _locationController.requestService();
  }else{
    return;
  }

  _permissionGranted = await _locationController.hasPermission();
  if(_permissionGranted==PermissionStatus.denied){
    _permissionGranted = await _locationController.requestPermission();
    if(_permissionGranted!=PermissionStatus.granted){
      return;
    }
  }

  _locationSubscription=_locationController.onLocationChanged.listen((LocationData currentLocation) {
    if(currentLocation.latitude != null && currentLocation.longitude != null){
      if(_currentPosition != LatLng(currentLocation.latitude!, currentLocation.longitude!)){
        animateCameraToPosition(LatLng(currentLocation.latitude!, currentLocation.longitude!));
      }
    }
    if(mounted){
      setState((){
        _currentPosition = LatLng(currentLocation.latitude!, currentLocation.longitude!);
        _updateUserMarker(_currentPosition!);
        _functions.updateUserLocation(user!.uid, widget.gameId, _currentPosition!);
      });
    }
  })
}

```

Rys. 5.41 Funkcja zarządzająca uprawnieniami oraz nasłuchiwaniami na zmiany lokalizacji użytkownika

Do aktualizacji markerów, czyli znaczników znajdujących się na mapie zastosowana została klasa Marker (Rys. 5.42). Przyjmuje atrybuty tj. Identyfikator, pozycja jako obiekt klasy **LatLng** oraz można ustawić dodatkowe informacje typu ikona czy adnotacja do znacznika.

```

void _updateUserMarker(LatLng position)async{
  Marker userMarker = Marker(
    markerId: userMarkerId,
    position: position,
    icon: markerIcon,
    infoWindow: InfoWindow(title: 'Your Location'),
  ); // Marker
  if(mounted){
    setState(() {
      markers[userMarkerId]=userMarker;
    });
  }
}

```

Rys. 5.42 Funkcja tworząca obiekt znacznika i przypisująca jego stan do tablicy

Funkcja **animateCameraToPosition** (Rys. 5.43) sprawdza czy kontroler mapy został zainicjowany, następnie za pomocą klasy **CameraPosition** ustawiane są wartości lokalizacji oraz zbliżenia na mapę. Kontroler oczekuje na wykonanie operacji

animateCamera, która powinna ustawić nową pozycję. Metoda jest wykonywana w momencie zmiany lokalizacji użytkownika w czasie rzeczywistym, pozwala to na łatwiejsze śledzenie swojego znacznika podczas poruszania.

```
Future<void> animateCameraToPosition(LatLng position)async{
    final GoogleMapController _controller = await mapController.future;
    CameraPosition _cameraPosition = CameraPosition(
        target: position,
        zoom: 15,
    );
    await _controller.animateCamera(
        CameraUpdate.newCameraPosition(_cameraPosition),
    );
}
```

Rys. 5.43 Funkcja zmieniająca położenie kamery ekranu w zależności od przyjętej lokalizacji

W momencie rozpoczęcia rozgrywki wywoływanych jest kilka funkcji, jedną z nich jest **fetchGameSet** (Rys 5.44). Pobiera numer identyfikacyjny z kolekcji “rooms”, a następnie znajduje i mapuje odpowiadający mu klucz z kolekcji “game_sets”, odbywa się to przy pomocy funkcji **getSetById**. Otrzymane dane są dzielone i przypisywane do dwóch zmiennych pomocniczych tj. numery id dokumentów filmów, czas trwania rozgrywki. Czas, który przypisany został jako mapa jest dzielony na dwie zmienne typu “Integer” odnoszące się do godzin oraz minut będących wartościami mapy. Następnie funkcja ustawia stan zmieniając czas trwania na ten będący przypisanym do konkretnego zestawu gry oraz suma filmów jakie są w zestawie zostaje przypisana do zmiennej pomocniczej. Kiedy już stany zostaną ustawione, rozpoczyna się start zegara oraz automatyczny reset tak, aby zegar mógł startować od wartości przyjętych z bazy danych. Kolejnym aspektem wizualnym dla użytkownika jest zainicjowanie markerów dla wszystkich filmów znajdujących się w zestawie gry wykonując operację w pętli “forEach” mapującą z bazy danych dane dokumentów filmów.


```

Future _fetchGameSet() async{
  gameSet = await _functions.getSetById(user!.uid,widget.gameId);
  List<dynamic> movieIds = gameSet.docIds;
  Map<dynamic,dynamic> time = gameSet.timeDuration;
  print("MAPA APPGAMESET: ${gameSet.docIds}");
  int h = int.parse(time['hours']);
  int m = int.parse(time['minutes']);
  setState(() {
    timeDuration= Duration(hours: h,minutes: m);
    toCollect = movieIds.length;
  });
  print("TO COLLECT: ${toCollect}");
  _startTimer();
  _resetTimer();

  movieIds.forEach((id) {
    _firestore.collection('films').doc(id).get().then((querySnapshot) {
      initMarker(querySnapshot.data(), id);
    });
  });
}

```

Rys. 5.44 Funkcja mapująca dane i ustawiająca startowe wartości gry terenowej

Zasada tworzenia markeru została już opisana, lecz w tym przypadku podczas inicjalizacji została zmieniona wartość atrybutu **infoWindow** (Rys. 5.45) na funkcjonalną. Kiedy użytkownik wyświetli okno z informacjami nad znacznikiem to ma możliwość kliknięcia w nie. Wywołany zostanie nawigator, który przeniesie użytkownika do strony przekazując zmapowane wartości za pomocą obiektu **AppFilm**, co pozwoli na wyświetlenie informacji o konkretnym filmie. Inicjalizowanie odbywa się w pętli `foreach` w innej funkcji, dlatego wartości znaczników muszą być dopisywane do tablicy markerów o ich specjalnej nazwie identyfikującej podczas aktualizacji stanu widżetu.

```

void initMarker(data, dataId) {
  var markerValue = dataId;
  LatLng latLng = LatLng(data['geopoint'].latitude, data['geopoint'].longitude);
  final MarkerId markerId = MarkerId(markerValue);
  final Marker marker = Marker(
    markerId: markerId,
    position: latLng,
    infoWindow: InfoWindow(title: data['title'], snippet: data['place'], onTap: () {
      print("$_currentPosition!-----$data['geopoint']");
      Navigator.push(context, MaterialPageRoute(builder: (context) => MoviePage(
        movie: AppFilm(
          uid: dataId,
          title: data['title'],
          director: data['director'],
          screenwriters: data['screenwriters'],
          filmGenre: data['film_genre'],
          cast: data['cast'],
          year: data['year'],
          rating: data['rating'],
          place: data['place'],
          imgPath: data['img_path'],
          geoPoint: data['geopoint'],
        ), // AppFilm
        currentPosition: _currentPosition!,
        inGame: true,
        code: widget.gameId,
      )))), // MoviePage // MaterialPageRoute // InfoWindow
  ); // Marker
  setState(() {
    markers[markerId] = marker;
  });
}

```

Rys. 5.45 Inicjalizacja obiektów klasy *Marker* i aktualizowanie stanu widżetu

Rozpoczęcie czasu trwania rozgrywki odbywa się poprzez ustawienie okresowego czasu trwania, za pomocą klasy **Duration** (Rys. 5.46) zegar jest w stanie zmniejszać swój czas co sekundę. Dodatkowym okresem jest dodanie ustawienia znacznika przeciwnika, który aktualizuje swój stan co minutę.

```

void _startTimer(){
  timer = Timer.periodic(Duration(seconds: 1), (_) => _addTime());
  updateTimer = Timer.periodic(Duration(minutes: 1), (Timer t) {
    _setOponentMarker();
  }); // Timer.periodic
}

```

Rys. 5.46 Funkcja ustawiająca stan początkowy zegarów

Wywołanie zmiany stanu lokalizacji markeru przeciwnika odbywa się przez sprawdzenie, który z użytkowników wykonuje operację. W ten sposób łatwo można pobrać lokalizację przeciwnika za pomocą funkcji **getOponentLocation**, która mapuje informację z bazy danych o wartości ostatniej lokalizacji użytkownika. Następnie znacznik przeciwnika jest inicjalizowany przyjmując tylko jako własności długość i szerokość geograficzną.

Funkcja **getCollectedCounter** (Rys. 5.47) tworzy strumień zawierający tylko wybrane elementy. Z kolekcji “rooms” pobierane są dane dokumentu o podanym id użytkownika i kodzie gry. Jeśli zwrócona zawartość nie jest pusta zostaje zwrócona liczba elementów listy zebranych miejsc przez użytkownika podczas rozgrywki. Głównym

przeznaczeniem działania tej funkcji jest zwracanie tej liczby w przypadku zmiany, a kiedy wartość elementów listy będzie równa wartości **toCollect** to za pomocą klasy **StreamBuilder** wyświetlony zostanie przycisk na stronie.

```
Stream<int> _getCollectedCounter(String uid) {  
  return _firestore.collection('rooms').where('uid',isEqualTo: uid).where('code',isEqualTo: widget.gameId).snapshots().map(  
    (querySnapshot) {  
      if(querySnapshot.docs.isEmpty){  
        List<dynamic> list = querySnapshot.docs.first['collected'];  
        return list.length;  
      }  
      return 0;  
    }  
  );  
}
```

Rys. 5.47 Funkcja zwracająca liczbę elementów listy przechowywanej w kolekcji *rooms* dla podanego id użytkownika

GameQuiz

Kiedy użytkownik wyświetli stronę z quizem, co odbywa się za pomocą wcześniej wymienionego przycisku, zostaną ustawione sprecyzowane pytania do gry terenowej.

Funkcja (Rys. 5.48) dodaje do listy z nazwami osób główną postać, o której jest quiz oraz 3 pozostałe nazwiska będące błędnymi odpowiedziami. Również zostaje zaimportowana lista profesji jakie może mieć dana osoba, obie listy są przemieszane za pomocą funkcji **shuffle**, która zamienia miejscami elementy. Wszystkie listy są wcześniej przygotowane w modelu **Answer**, tak aby łatwo można było podawać nazwę oraz wartość true lub false w zależności od prawdziwości odpowiedzi.

```
void setQuestionList(){  
  nameList.add(  
    Answer(widget.gameSet.name, true),  
  );  
  nameList.add(  
    Answer("Andrzej Wajda", false),  
  );  
  nameList.add(  
    Answer("Roman Polański", false),  
  );  
  nameList.add(  
    Answer("Tomasz Kot", false),  
  );  
  widget.gameSet.profession.forEach((key,value) {  
    profList.add(  
      Answer(key, value)  
    );  
  });  
  nameList.shuffle();  
  profList.shuffle();  
}
```

Rys. 5.48 Funkcja ustawiająca i mieszająca wartości obiektu *Answer* dla wybranego quizu

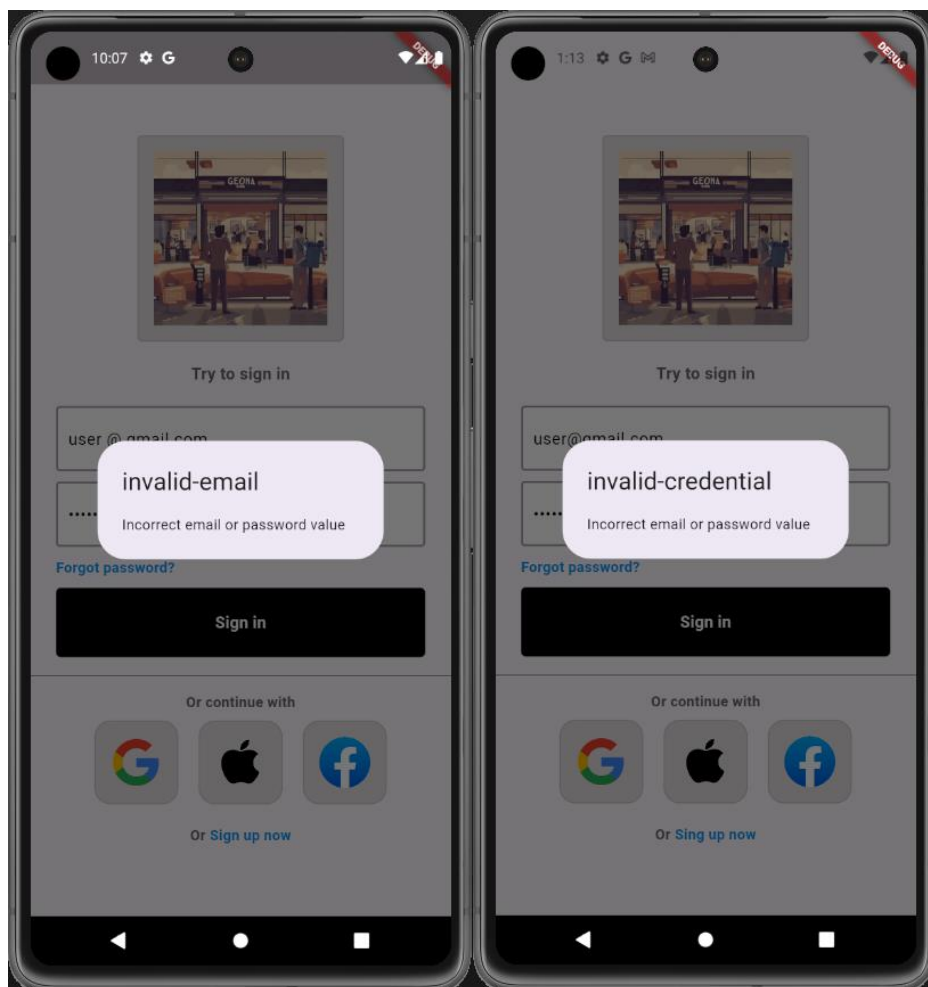
5.6. Problemy implementacyjne

Podczas projektowania bazy danych pod kątem gry terenowej było założenie, w którym baza miała się głównie opierać na Realtime Database. Jest to narzędzie, w którym w łatwy sposób za pomocą referencji można manipulować przechowywanymi danymi. Początkowo lokalizacja użytkownika oraz przeciwnika czy informacje dotyczące zbieranych miejsc w rozgrywce przechowywane były w referencji “users”, lecz niestety w przypadku aktualizacji jakichkolwiek wartości powodowało to problemy występujące na mapie. Mianowicie w przypadku aktualizacji mapa Google odświeżała się za każdym razem co powodowało rozpoczęcie początkowego stanu przykładowo resetowanie zegara. W takim przypadku stosując nasłuchiwanie na lokalizacji użytkownika, które wywołuje aktualizację w bazie danych długości i szerokości geograficznej w momencie zmiany lokalizacji było nieoptymalne i powodowało niedogodności. Rozwiązaniem okazało się korzystanie jedynie z kolekcji Firestore oraz ograniczenie zapisu i aktualizacji danych o określony czas, którym jest minuta.

5.7. Testowanie

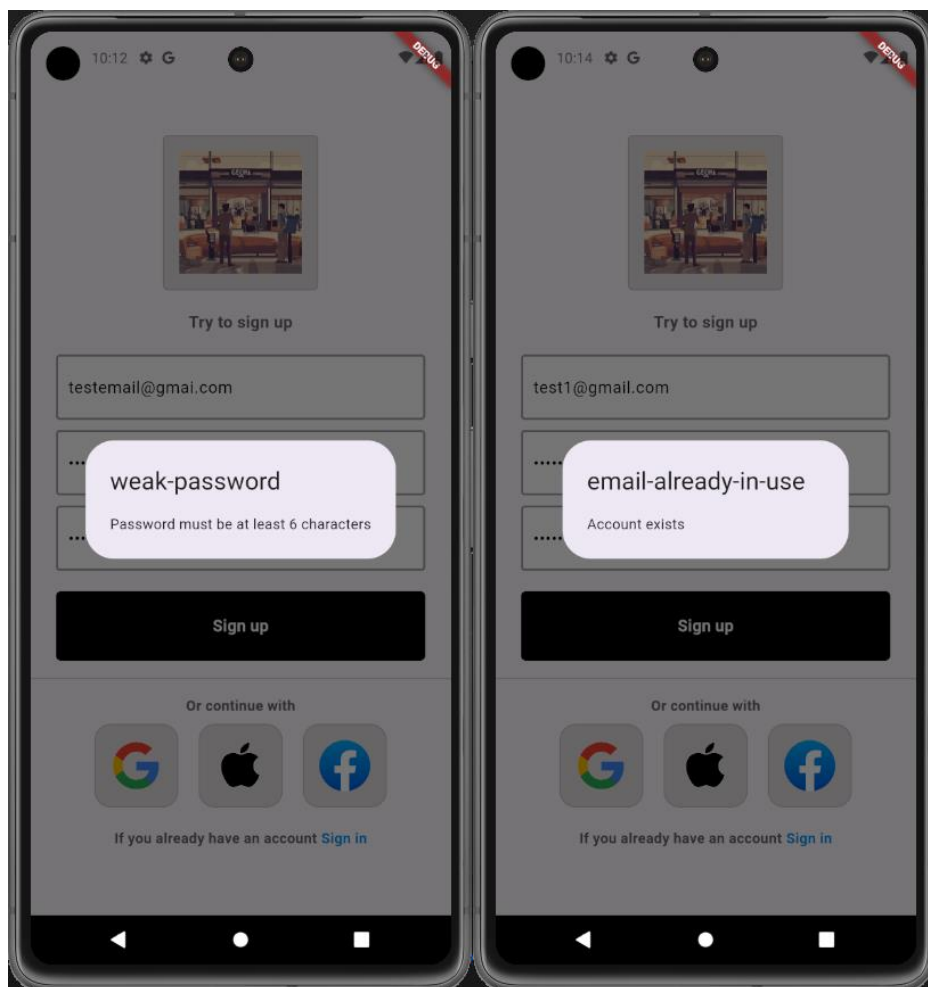
W projekcie zostały przeprowadzone testy manualne, aby przedstawić jak aplikacja jest odporna na niepoprawne dane wprowadzane przez użytkowników. W tym celu najważniejszym punktem jest zabezpieczenie dostępu do aplikacji oraz elementów w przypadku rejestracji i logowaniu użytkownika.

W przypadku logowania jeśli użytkownik błędnie wprowadzi adres email, w sposób niepoprawny do budowy adresu, czyli np. użycie spacji między znakami kluczowymi, to aplikacja zwróci komunikat (Rys. 5.49). Informuje on o niepoprawności składni adresu email lub hasła. Jeśli użytkownik wprowadzi adres email, który nie istnieje w bazie danych zostanie wyświetlony komunikat „Invalid-credential” (Rys 5. 50).



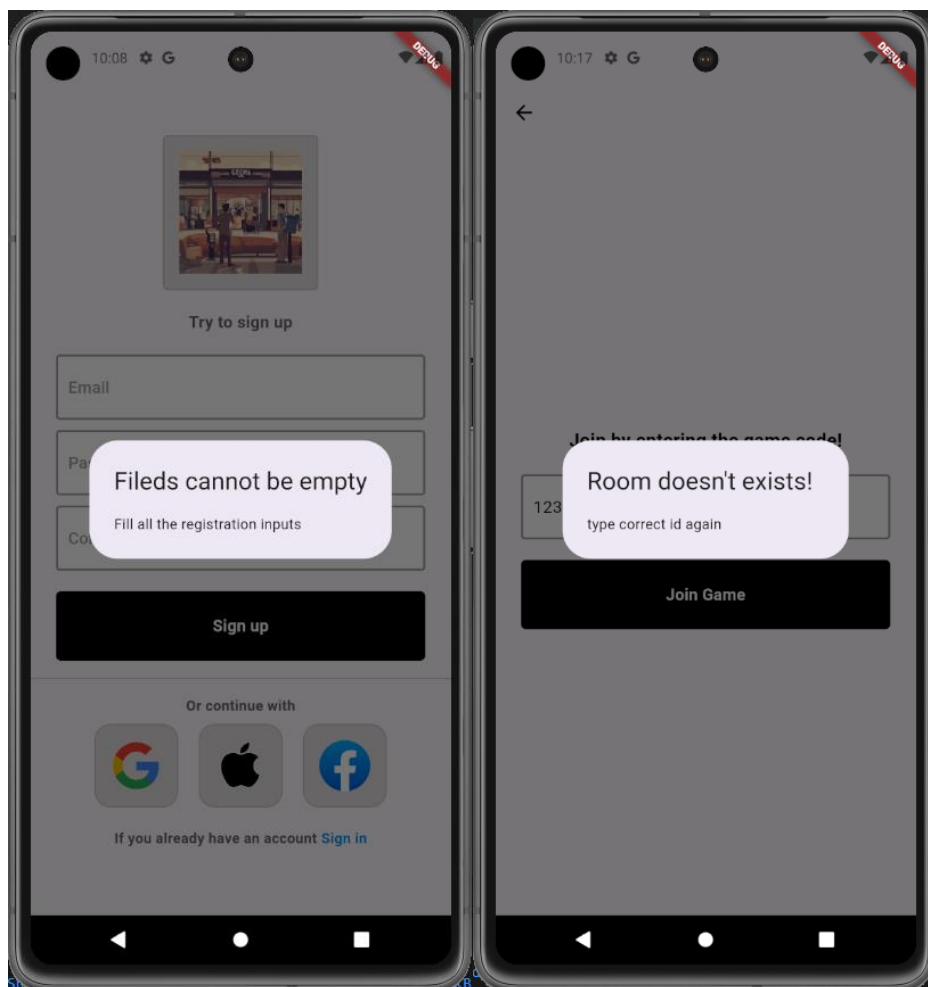
Rys. 5.49 Niepoprawna składnia danych Rys. 5.50 Konto nie istnieje

W przypadku rejestracji istnieją dodatkowe zabezpieczenia w postaci polityki haseł, która została ustawiona w Firebase i wymaga minimum 6 znaków podanych w haśle (Rys. 5.51). Jeśli użytkownik w momencie tworzenia konta spróbuje wpisać niepoprawnie adres email to zostanie wyświetlony komunikat „invalid-email” (Rys. 5.49). Kiedy zdarzy się sytuacja, że zostanie wpisany adres email, który znajduje się już w bazie danych to użytkownik zostanie poinformowany przez odpowiedni komunikat „email-already-in-use” (Rys. 5.52).



Rys. 5.51 Hasło nie spełnia wymagań Rys. 5.52 Email istnieje w bazie danych

Istnieje również zabezpieczenie w przypadku nie wypełnienia jakiegoś pola przez użytkownika w formularzu rejestracji (Rys. 5.53), może to prowadzić do błędów przy tworzeniu konta. Przy formularzach dla zalogowanych użytkowników istnieją również takie zabezpieczenia, żeby nie wysyłać pustych tekstów. Ostatnim zabezpieczeniem pól tekstowych jest obsługa błędnego kodu gry, który może wprowadzić użytkownik. Kiedy w bazie danych nie istnieje dany pokój gry to zostaje wyświetlony komunikat (Rys. 5.54).

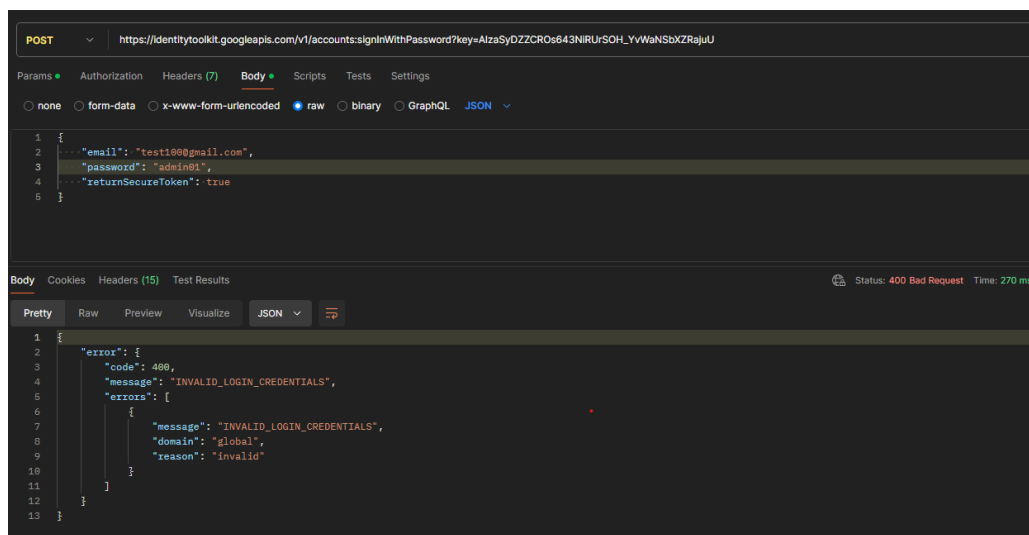


Rys. 5.53 Pola nie mogą być puste Rys. 5.54 Pokój gry nie istnieje

Ważnym elementem testowania jest również testowanie backendu, w tym wypadku jest to testowanie API bazy danych. Do sprawdzania poprawności wprowadzania oraz pozyskiwania danych z klucza API został wykorzystany Postman, którego komunikaty zaistniałe na poniższych rysunkach zostały opisane w tabeli (tabela 5.1). Za pomocą funkcji w adresie URL „signInWithPassword” możliwe jest testowanie logowania użytkownika do systemu. Podanie adresu email w metodzie POST, który nie istnieje w bazie danych powoduje otrzymanie komunikatu 400 (Rys. 5. 55).

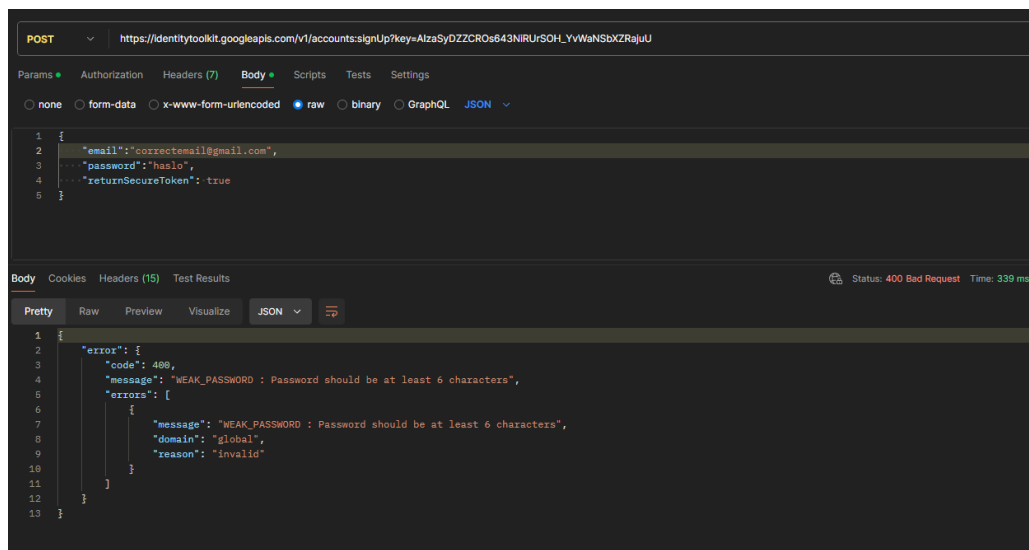
Tabela 5.1 Komunikaty pojawiające się podczas testowania backendu

Numer komunikatu odpowiedzi	Komunikat
200	„OK” – serwer znalazł żądany obiekt
400	„Bad request” – nieprawidłowe żądanie



Rys. 5.55 Błędne dane logowania

Aby przetestować niepoprawną rejestrację użytkownika zostało wprowadzone hasło które nie spełnia wymagań polityki haseł (Rys 5.56), do tego użyto funkcji w linku URL „**signUp**”. Dodatkowo wprowadzono błędny adres email, dla którego zostaje wyświetlony podobny komunikat (Rys 5.55).



Rys. 5.56 Nieprawidłowe hasło przy rejestracji

W projekcie została użyta symulacja obciążeniowa serwera, do tego posłużyło narzędzie Locust, które polega na stworzeniu roju użytkowników i zasymulowanie ich żądań. Do przeprowadzenia testu zostały użyte dwie metody GET oraz POST, użytkownicy odczytywali zawartości dokumentów z Firestore oraz zapisywali w konkretnej kolekcji dokument (Rys. 5.57).



Locust

HOST

https://firestore.googleapis.com

STATUS

STOPPED

RPS

327.3

FAILURES

1%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

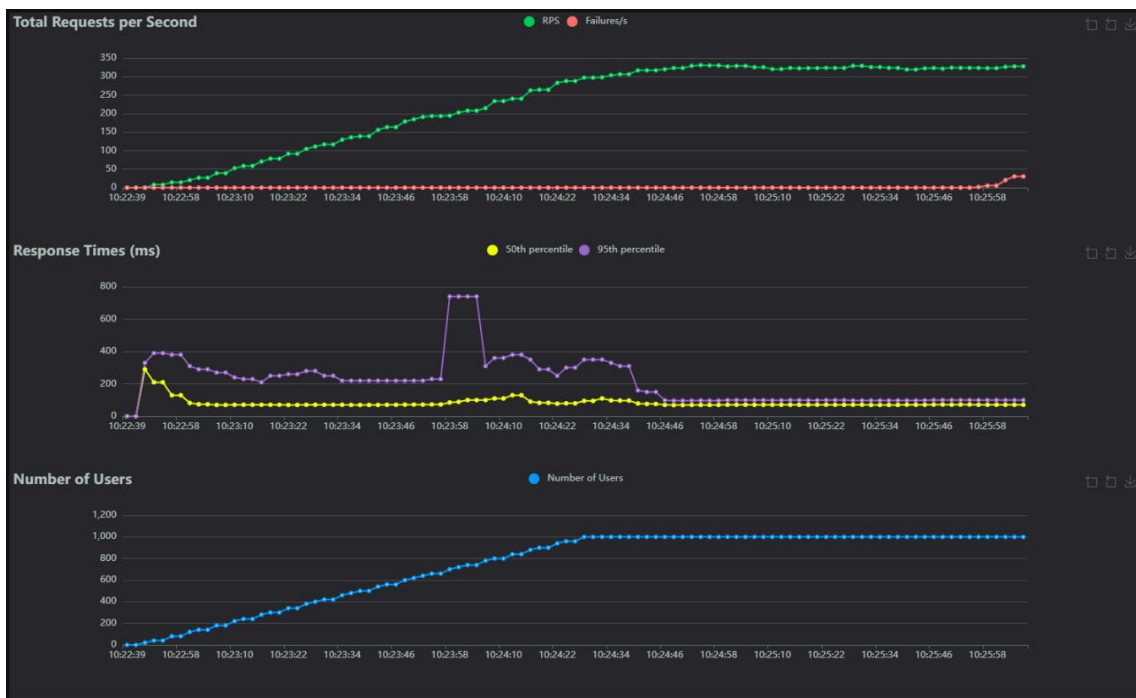
DOWNLOAD DATA

LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/v1/projects/filmapp-f3c71/databases/(default)/documents/forms/00iQF9YVMpzVZ00fCeLn?key=a9895f684f4962c0719d77c107314a124282059	23739	0	79	240	440	99.75	42	1552	379	158.4	0
POST	/v1/projects/filmapp-f3c71/databases/(default)/documents/forms?key=a9895f684f4962c0719d77c107314a124282059	23932	515	71	210	410	92.18	33	1367	308.56	168.9	35.5
	Aggregated	47671	515	73	220	430	95.95	33	1552	343.64	327.3	35.5

Rys. 5.57 Utworzenie roju do symulacji zachowań użytkowników

Jak można zauważyć na powyższym rysunku średni czas dla odczytu wynosił około **100ms** a dla zapisu około **92ms**. W przypadku zapisu po pewnym czasie nastąpiły błędy co jest widoczne na wykresie (Rys. 5.58), spowodowane jest to przepełnieniem kolekcji Firestore i odrzucaniem żądań. Test został przeprowadzony na 1000 użytkowników i na tworzeniu żądań co **10 sekund**, po upływie 3,5min przez ilość żądań występowały błędy.



Rys. 5.58 Wykres przedstawiający czasy, liczbę żądań oraz ilość użytkowników

6. Podsumowanie

W niniejszej pracy opisany został proces budowania aplikacji mobilnej zaczynając od wyboru niezbędnych do realizacji postawionego celu. Za pomocą przypadków użycia i wymagań zostały opisane funkcjonalności użytkowników w systemie oraz jakie cechy ma aplikacja. Następnie projekt aplikacji pozwolił zwizualizować możliwości poruszania się użytkownika po systemie i wykorzystywanie interfejsu. Implementacja pomogła autorowi zrozumieć zespojenie ze sobą technologii oraz ukazała jak stworzone funkcje wpływają na działanie systemu.

6.1. Podsumowanie pracy

Celem pracy było stworzenie projektu aplikacji mobilnej działającej w oparciu o system **Android** i **iOS**, która będzie umożliwiała użytkownikom zwiedzanie miejsc popularnych scen filmowych na terenie Krakowa i okolic. Podstawowym założeniem było również pozyskiwanie lokalizacji użytkownika oraz wyświetlanie pobliskich znaczników za pomocą map. Opracowanie aplikacji wiązało się ze zgromadzeniem informacji na temat filmów jakie były tworzone na terenie Krakowa, co okazało się dość sporym wyzwaniem i pokazało, ile tak naprawdę możliwości może mieć projekt. Aplikacja łączy ze sobą strefę turystyczną oraz zainteresowanie filmami, jest to nowe rozwiązanie na rynku i może stanowić ważny element promujący miasto jak i sztukę filmową. Kluczowe funkcjonalności udało się zrealizować w sposób podstawowy umożliwiając użytkownikom logowanie, rejestrację czy dostęp do zasobów konta. Została utworzona gra terenowa dzięki której użytkownicy mogą rywalizować ze sobą, odwiedzając różne miejsca kultury przedstawione w scenach filmowych. Dzięki odpowiedniemu wyborowi bazy danych wszystkie dane są wyświetlane i aktualizowane w czasie rzeczywistym co ułatwia korzystanie z aplikacji.

6.2. Dalszy plan rozwoju aplikacji

Głównym motywem, który powinien być rozwijany jest to rozszerzenie możliwości gry terenowej jak i bazy informacji o filmach o inne polskie miejscowości. Wiąże się to ze zbieraniem informacji oraz przeprowadzeniem badań w różnych lokalizacjach. Dodatkowo należałoby rozwinąć możliwość wpływu użytkownika na aplikację oraz rozwinąć funkcjonalności związane z administrowaniem systemem. Aby ułatwić wprowadzanie nowych miejsc na mapę powinien zostać utworzony formularz przyjmujący informację od użytkownika na temat miejsca oraz filmu jaki był w nim nagrywany. Dzięki różnym uprawnieniom administrator mógłby nadzorować proces dodawania nowych lokalizacji i filmów poprzez odrzucanie lub przyjmowanie danego formularza. Aplikacja powinna posiadać aktualne zdjęcia miejsc scen filmowych tak, aby jak najlepiej zobrazować użytkownikowi miejsca, które odwiedza.

Spis ilustracji

Rys. 1.1 Ekran główny aplikacji	Rys. 1.2 Mapa usług w okolicy	7
Rys. 1.3 Aplikacja do tworzenia gier terenowych [3]		8
Rys. 2.1 Google Pixel 7a	Rys. 2.2 Google Pixel 3a	14
Rys. 2.3 Wyszczególnienie platform obsługiwanych przez Android Studio [13].....		15
Rys. 2.4 Badanie wykorzystywania edytorów w firmach [15].....		16
Rys. 3.1 Diagram przypadków użycia		19
Rys. 4.1 Ekran logowania	Rys. 4.2 Ekran rejestracji.....	25
Rys. 4.3 Ekran główny	Rys. 4.4 Ekran dołączania do rozgrywki.....	26
Rys. 4.5 Widok jednego użytkownika	Rys. 4.6 Widok dwóch użytkowników	27
Rys. 4.7 Znacznik przeciwnika	Rys. 4.8 Znaczniki miejsc.....	28
Rys. 4.9 Informacje o filmie	Rys. 4.10 Wyświetlenie przycisku.....	29
Rys. 4.11 Komunikat do zatwierdzenia	Rys. 4.12 Komunikat o opuszczeniu rozgrywki	30
Rys. 4.13 Przycisk kontynuacji	Rys. 4.14 Ekran startowy Quizu.....	31
Rys. 4.15 Komunikat spróbuj ponownie	Rys. 4.16 Ukończenie quizu.....	32
Rys. 4.17 Lokalizacja użytkownika	Rys. 4.18 Znacznik z informacją o filmie.....	33
Rys. 4.19 Profil użytkownika	Rys. 4.20 Widok rankingu.....	34
Rys. 4.21 Ekran ustawień	Rys. 4.22 Formularz zmiany hasła	35
Rys. 4.23 Usunięcie konta	Rys. 4.24 Formularz zgłoszenia błędu	36
Rys. 4.25 Ustawienia Administratora	Rys. 4.26 Panel administratora	37
Rys. 5.1 Panel uwierzytelniania jako jedna z funkcjonalności Firebase		39
Rys. 5.2 Struktura kolekcji w Firestore		39
Rys. 5.3 Przykładowa struktura zbudowanego dokumentu w kolekcji users		40
Rys. 5.4 Struktura referencji zawartych w bazie RealtimeDatabase		40
Rys. 5.5 Zestawienie bibliotek oraz narzędzi użytych w projekcie znajdujących się w pliku pubspec.yaml		41
Rys. 5.6 Kompletna struktura plików utworzonych w projekcie		42
Rys. 5.7 Klasa AppFilm	Rys. 5.8 Klasa AppGameInfo.....	43
Rys. 5.9 Funkcja logowania za pomocą adresu e-mail oraz hasła.....		44
Rys. 5.10 Logowanie za pomocą uwierzytelniania z kontem Google.....		45
Rys. 5.11 Logowanie za pomocą uwierzytelniania z kontem Facebook		45
Rys. 5.12 Rejestracja użytkownika z wykorzystaniem adresu e-mail oraz hasła.....		46
Rys. 5.13 Funkcja usuwająca konto użytkownika za bazy danych		47
Rys. 5.14 Dodanie użytkownika do kolekcji users		47
Rys. 5.15 Funkcja zwracająca id dokumentu jako wartość tekstową.....		48
Rys. 5.16 Funkcja aktualizująca uprawnienia użytkownika.....		48

Rys. 5.17 Funkcja zwracająca wynik zapytania jakim jest posortowany rankingużytkowników	48
Rys. 5.18 Lista klas w których znajdują się zawartości stron z menu dolnego	49
Rys. 5.19 Przypisywanie elementu pochodnego poprzez wybrany indeks	49
Rys. 5.20 Atrybut klasy BottomNavigationBar wraz z formatowaniem	50
Rys. 5.21 Stylizacja komponentu przycisków	50
Rys. 5.22 Stylizacja komponentu ikon	51
Rys. 5.23 Stylizacja komponentu pola tekstowego	52
Rys. 5.24 Budowa komponentu kontenera wywoływanego na konkretnej wartości z bazy danych.....	52
Rys. 5.25 Funkcja zwracająca obiekt AppUser	53
Rys. 5.26 Funkcja aktualizująca pole visited w bazie danych dla kolekcji users.....	54
Rys. 5.27 Funkcja aktualizująca pole visited w bazie danych dla kolekcji users.....	54
Rys. 5.28 Klasa GestureDetector wraz z stylizacją	54
Rys. 5.29 Klasa wraz z metodami komponentów tytułowych.....	55
Rys. 5.30 Klasa przedstawiania danych w tabeli.....	55
Rys. 5.31 Klasa Visibility wyświetlająca lub ukrywająca element na stronie	55
Rys. 5.32 Funkcja stosująca wzór Haversine do obliczania odległości między punktami na ziemi.....	57
Rys. 5.33 Klasa GamePage wyświetlająca możliwości w jakie użytkownik może dołączyć do rozgrywki.....	57
Rys. 5.34 Funkcja tworząca referencje do bazy danych i nawigująca do kolejnej strony	58
Rys. 5.35 Funkcja generująca losowy ciąg cyfr oraz liter składający się na 6 znakowy kod	58
Rys. 5.36 Funkcja odpowiadająca za dodawanie referencji użytkownika do bazy danych oraz wyświetlenie strony pokoju	59
Rys. 5.37 Nasłuchiwanie na zmiany w referencjach na węźle rooms o podanym kodzie gry	60
Rys. 5.38 Nasłuchiwanie na zmiany w referencjach na węźle users dla pobranego id użytkownika oraz sprawdzanie gotowości	60
Rys. 5.39 Funkcja dodająca informacje o utworzonym pokoju do bazy danych w kolekcji rooms	61
Rys. 5.40 Usunięcie historii użytkowników oraz wszystkich elementów odnoszących się do pokoju rozgrywki.....	62
Rys. 5.41 Funkcja zarządzająca uprawnieniami oraz nasłuchiowaniem na zmiany lokalizacji użytkownika	63
Rys. 5.42 Funkcja tworząca obiekt znacznika i przypisująca jego stan do tablicy	63

Rys. 5.43 Funkcja zmieniająca położenie kamery ekranu w zależności od przyjętej lokalizacji.....	64
Rys. 5.44 Funkcja mapująca dane i ustawiająca startowe wartości gry terenowej	65
Rys. 5.45 Inicjalizacja obiektów klasy Marker i aktualizowanie stanu widżetu	66
Rys. 5.46 Funkcja ustawiająca stan początkowy zegarów	66
Rys. 5.47 Funkcja zwracająca liczbę elementów listy przechowywanej w kolekcji rooms dla podanego id użytkownika	67
Rys. 5.48 Funkcja ustawiająca i mieszająca wartości obiektu Answer dla wybranego quizu.....	67
Rys. 5.49 Niepoprawna składnia danych Rys. 5.50 Konto nie istnieje	69
Rys. 5.51 Hasło nie spełnia wymagań Rys. 5.52 Email istnieje w bazie danych.....	70
Rys. 5.53 Pola nie mogą być puste Rys. 5.54 Pokój gry nie istnieje.....	71
Rys. 5.55 Błędne dane logowania.....	72
Rys. 5.56 Nieprawidłowe hasło przy rejestracji	72
Rys. 5.57 Utworzenie roju do symulacji zachowań użytkowników	73
Rys. 5.58 Wykres przedstawiający czasy, liczbę żądań oraz ilość użytkowników	73

Bibliografia

1. Białek Artur, *Czy pierwszy komputer powstał w USA, w Niemczech czy w Anglii? Spory o to trwają do dziś.*, 2023.
<https://www.national-geographic.pl/artykul/pierwszy-komputer-jak-wyglada-historia-komputera-na-swiecie> [data dostępu: 2024-07-30]
2. *Pierwszy telefon komórkowy – kiedy powstał i jak wyglądał?*
<https://www.morele.net/wiadomosc/pierwszy-telefon-komorkowy-kiedy-powstal-i-jakwygladal/16969/> [data dostępu: 2024-07-30]
3. *Jak działa ActionTrack?*
<https://mobilnegrymiejskie.pl/action-track/> [data dostępu: 2024-07-30]
4. Wantoch-Rekowski Roman, *Android w praktyce: projektowanie aplikacji*, Wydawnictwo naukowe PWN S.A, Warszawa 2014, s.5., ISBN 978-83-01-17949-6
https://helion.pl/eksiazki/android-w-praktyce-projektowanie-aplikacji-roman-wantochrekowski,e_1o05.htm [data dostępu: 2024-07-30]
5. *Co to jest iOS?*
<https://asaricrm.com/slownik-crm/ios/> [data dostępu: 2024-08-10]
6. Pióro Monika, *IOS vs. Android – porównanie obu systemów operacyjnych*, 2023. <https://homescreen.pl/pl/ios-vs-android-porownanie-obu-systemow-operacyjnych-.htm> [data dostępu: 2024-07-30]
7. Shapovalov Eugene, *Odkrywanie korzyści płynących z programowania full-stack z Flutter*, 2023
8. Moskala Marcin, *Czym jest Kotlin*, rozdział z książki *Kotlin Essentials*
<https://kt.academy/pl/article/kfde-whats-kotlin> [data dostępu: 2024-07-30]
9. Kozon Tomasz, *Swift*
<https://boringowl.io/tag/swift> [data dostępu: 2024-07-30]
10. Apple Inc., *The Swift Programming Language – Swift 5.7 Edition*, s.3 2022
<https://www.appsdissected.com/wp-content/uploads/2022/11/SwiftProgrammingLanguage57.pdf> [data dostępu: 2024-07-30]
11. *Why did we build Visual Studio Code?*, 2024
<https://code.visualstudio.com/docs/editor/whyvscode> [data dostępu: 2024-07-30]
12. Szewczyk Karolina, *Testowanie aplikacji mobilnych za pomocą Android Studio*, TestArmy Group S.A.

- <https://testuj.pl/blog/testowanie-aplikacji-mobilnych-za-pomoca-android-studio/>
[data dostępu: 2024-07-30]
13. Oracle, *Tworzenie urzędzeń wirtualnych i zarządzanie nimi*, 2024
<https://developer.android.com/studio/run/managing-avds?hl=pl> [data dostępu: 2024-07-30]
 14. Kulsreshtha Arpit, *iOS 17 App Development for beginners*, s.1, 2024 BPB Online, ISBN 978-93-55515-858
 15. *IDEs And Text Editors*
<https://6sense.com/tech/ides-and-text-editors> [data dostępu: 2024-07-30]
 16. Mamczur Mirosław, *Jak policzyć odległość między dwoma punktami?*
<https://mirosławmamczur.pl/jak-zmienic-adres-w-wspolrzedne-geograficzne-i-sprawdzic-np-najblizsze-restauracje-openstreetmap/> [data dostępu: 2024-08-10]
 17. Griffiths Dawn, *Kotlin. Rusz głową!*, Helion, 2020, s.2-3, ISBN 978-83-283-5869-0
 18. *Rodzaje wymagań*
<https://wolski.pro/2015/02/rodzaje-wymagan/> [data dostępu: 2024-08-07]
 19. Kettle Simon, *Distance on a sphere: The Haversine Formula*.
<https://community.esri.com/t5/coordinate-reference-systems-blog/distance-on-a-sphere-the-haversine-formula/ba-p/902128> [data dostępu: 2024-08-10]